

Clustered Randomized Smoothing for Stochastic Prediction Functions

Eduardo Figueiredo^{*,1} Frederik Mathiesen^{*,1} Julian Schumann²
Jens Kober³ Arkady Zgonnikov² Luca Laurenti^{1,4}

¹Delft Center for Systems and Control, Delft University of Technology

²Cognitive Robotics, Delft University of Technology ³University of Stuttgart ⁴AI4I

{e.figueiredo, f.b.mathiesen, j.f.schumann, a.zgonnikov, l.laurenti}@tudelft.nl

Abstract

Modern stochastic predictors can model rich, multi-modal outcome distributions. However, this expressive power comes with challenges in ensuring robust predictions—a critical requirement in safety-critical domains. Randomized smoothing is a leading technique for improving robustness, particularly against adversarial perturbations. Yet, in stochastic multi-modal regression settings, randomized smoothing often fails due to mode collapse, yielding averaged predictions that do not reflect the underlying distribution. To address this limitation, we propose clustered α -smoothing, a framework that (1) partitions noisy samples using an arbitrary clustering algorithm, (2) applies α -smoothing locally within each cluster, and (3) combines the resulting predictions into a mixture distribution. By interpreting the smoothing distribution as a mixture of α -smoothers, we derive a lower bound on the probability that the smoothed prediction lies within a union of compact regions corresponding to distinct modes. We empirically evaluate our framework on two benchmarks, demonstrating substantial improvements over state-of-the-art methods. In stochastic trajectory prediction on a driving simulator dataset, our approach achieves, on average, a 27% lower Wasserstein distance to the ground-truth distribution compared to α -smoothing. In quadrotor control, where modes correspond to distinct feasible paths to a target, our method reduces the collision rate by 81% relative to the state-of-the-art randomized smoothing.

1 Introduction

Stochastic predictors, such as variational autoencoders [10], normalizing flows [28], or Bayesian neural networks [43], are capable of modeling rich, multi-modal outcome distributions [21, 12]. However, this expressive power comes with challenges in ensuring robust predictions – a fundamental requirement in safety-critical applications such as electrical grids, healthcare, robotics, or autonomous driving, where this type of brittle behavior can have catastrophic consequences [9, 24, 23, 37]. A particular concern is vulnerability to adversarial attacks [15, 22, 35, 29, 6], in which carefully crafted, small perturbations of the input cause significant deviations in the network’s output. To address this issue, a wide range of methods has been developed for formally verifying desired input–output properties of neural networks, including mixed-integer programming [16], abstract interpretation [13], linear relaxations with branch-and-bound [48, 45], satisfiability modulo theory solvers [44, 11], and reachability analysis [41, 1]. Despite their strong guarantees, these methods are often computationally expensive, which limits their practical applicability.

Randomized smoothing is a scalable alternative for certifying robustness [20, 17, 8]. Given a base predictor, randomized smoothing injects noise into the input and averages the resulting predictions, yielding a smoothed predictor with certified robustness guarantees. While most prior work focuses on classification [20, 17, 8], recent efforts extend these ideas to regression [7, 31, 30, 36]. In this setting,

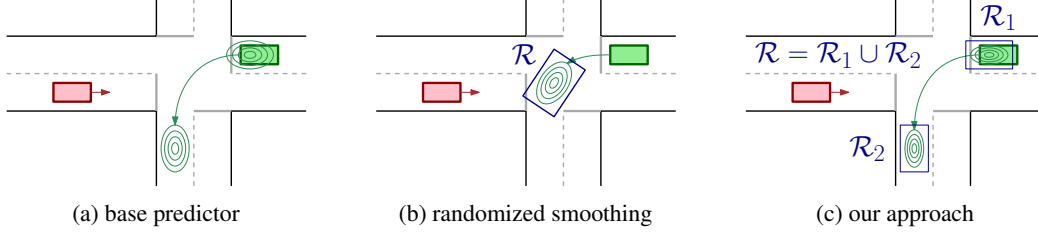


Figure 1: Conceptual illustration of mode collapse in randomized smoothing. In (a), an ego vehicle (red) is predicting the trajectory of another vehicle (green) at a cross junction. The stochastic predictor outputs a distribution with two modes: *turning left* before the ego vehicle passes the intersection, or *waiting* for it to pass before initiating the turn. In (b), current randomized smoothing methods (e.g., [30]) would produce an averaged prediction, failing to capture any of the two modes. Instead, in (c), our approach is to smooth each mode individually and combine as a mixture model, which preserves the modes corresponding to the two tactical behaviors. Sets $\mathcal{R}$ represent the sets that can be certified with each method.

given a base predictor $h : \mathbb{R}^d \rightarrow \mathbb{R}^q$ and a Gaussian noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 I)$ with $\sigma^2 > 0$, the smoothed predictor is commonly defined as (some variant of) $\mathcal{H}(x) = \mathbb{E}_{\epsilon} [h(x + \epsilon)]$. However, this approach is fundamentally limited for stochastic neural network predictors. In these models, predictions are often inherently multi-modal [10, 28, 43], yet randomized smoothing collapses this structure by averaging over noisy outputs, yielding a unimodal and potentially uninformative estimate (Figure 1). This limitation is particularly problematic in safety-critical applications where multi-modality is essential, such as trajectory prediction [27, 26, 4, 37] or failure prediction [33, 49].

To address this limitation and capture the multi-modal behavior of the base predictor, we propose *clustered α -smoothing*. Given an input x and a set of noise samples $\{\epsilon_1, \dots, \epsilon_N\}$, our method (i) clusters the prediction samples $\{h_{\mathbf{w}}(x + \epsilon_1), \dots, h_{\mathbf{w}}(x + \epsilon_N)\}$, (ii) trims α outlier samples per cluster, and (iii) smooths locally within each cluster. The clusters are then combined according to a mixture distribution. By combining the modes/clusters as mixture model rather than averaging, we avoid mode collapse, and thus, the limitation of the previous state-of-the-art. Further, the per-mode α -trimming allows greater control over outliers. We prove by classical probability theory results and statistical methods that the output of the smoothed predictor belongs to a coverage region $\mathcal{R}$ with high probability for all inputs in a radius r . The guaranteed coverage region $\mathcal{R}$ is provided as the union of the cluster-wise coverage regions $\mathcal{R}_m$, which are regions containing the samples of each cluster with high probability (see Figure 1), and the probability of the prediction belonging to the coverage region is provided both individually and jointly for all clusters. Our framework is agnostic to the clustering algorithm and provides flexibility in how to determine the number of clusters and sample assignment. We demonstrate the framework on two benchmarks: (1) stochastic multi-modal trajectory prediction in traffic (Figure 1), a key prerequisite for safe and efficient automated driving, and (2) robustification of a multi-modal deep RL quadrotor controller, where the framework helps improve robustness while preserving multiple feasible navigation strategies around obstacles.

In summary, our main contributions are: (i) we introduce a framework for randomized smoothing for multi-modal stochastic predictors via clustering and α -trimming, (ii) we prove that the output of the clustered α -smoothed predictor with high probability falls within a small coverage region per cluster for all inputs in a radius, and (iii) we demonstrate the effectiveness of our framework in benchmarks from stochastic trajectory prediction in traffic and a multi-modal RL controller for a quadrotor, showing improved performance compared to state-of-the-art.

2 Preliminaries on randomized smoothing

Notation. For a vector $x \in \mathbb{R}^d$, we denote by $x^{(i)}$ its i -element. Given $N \in \mathbb{N}_{>0}$, the set $\{1, \dots, N\}$ is denoted by $[N]$. I denotes the identity matrix. The Pontryagin difference between sets A, B is denoted by $A \ominus B$. For a set $\mathcal{X} \subseteq \mathbb{R}^d$, the indicator function for $\mathcal{X}$ is denoted as $\mathbf{1}_{\mathcal{X}}(x) := 1$ if $x \in \mathcal{X}$; otherwise 0. Given a Borel measurable space $\mathcal{X} \subseteq \mathbb{R}^d$, we denote by $\mathcal{B}(\mathcal{X})$ the Borel sigma algebra over $\mathcal{X}$ and by $\mathcal{P}(\mathcal{X})$ the set of probability distributions on $\mathcal{X}$. For a random variable $\mathbf{x}$ taking values in $\mathcal{X}$, $\mathbf{x} \sim \mathbb{P}_{\mathbf{x}} \in \mathcal{P}(\mathcal{X})$ represents the probability measure associated to $\mathbf{x}$. Let $f_{n,k}(p) = \binom{n}{k} p^k (1-p)^{n-k}$ denote throughout the paper the probability mass function of a binomial random variable with parameters n and k . For a vector $x \in \mathcal{X}$, the k -th order statistic of x is denoted by $x_{(k)}$. Further, we define a stochastic predictor as follows.

Definition 1 (Stochastic Predictor). *Consider a deterministic function $h_w : \mathcal{X} \rightarrow \mathcal{Y}$ with $\mathcal{X} \subseteq \mathbb{R}^d$ and $\mathcal{Y} \subset \mathbb{R}^q$, where $w \in \mathbb{R}^m$ is a parameter vector. Then, for $x \in \mathcal{X}$ and a possibly input dependent random vector $\mathbf{w} \sim \mathbb{P}_{\mathbf{w}|x}$ on $\mathbb{R}^m$, the stochastic prediction induced by h_w and $\mathbf{w}$ at x is the random variable $h_{\mathbf{w}}(x)$.*

Our definition of stochastic predictor is sufficiently general to encompass important stochastic neural models such as Bayesian neural networks (BNNs) [43], variational autoencoders (VAEs) [10], and normalizing flows [28]. In what follows, for simplicity, we assume that the distribution of $\mathbf{w}$ does not depend on the input x . Accordingly, we write $\mathbb{P}_{\mathbf{w}}$ instead of $\mathbb{P}_{\mathbf{w}|x}$. However, our methods in Sections 3 and 4 also apply to the case where $\mathbf{w}$ depends on the input x .

Randomized smoothing. Although the literature on randomized smoothing has been extensively developed for classification problems [8, 19, 46, 18], recent work has recently sparked interest in regression settings [7, 31, 30]. Given a stochastic predictor $h_{\mathbf{w}}$ and $x \in \mathcal{X}$, randomized smoothing can be viewed as the transformation $\mathcal{H}(x) = \mathbb{E}_{\varepsilon}[h_{\mathbf{w}}(x + \varepsilon)]$, i.e., the original stochastic function is replaced by its expectation w.r.t. noisy perturbations around x . In practice, computing $\mathcal{H}(x)$ in closed-form is often infeasible [14], thus commonly replaced by its sampled approximation $\mathcal{H}_N(x) = \frac{1}{N} \sum_{i=1}^N h_{\mathbf{w}}(x + \varepsilon_i)$, where $\{\varepsilon_1, \dots, \varepsilon_N\}$ are i.i.d. samples from the distribution of ε . However, as discussed in [30], such smoother presents a major weakness: the smoothed function is vulnerable to outliers in the prediction, with a single sample being capable of pushing the smoothed output beyond the desired range $\mathcal{R}$. Therefore, [30] proposes a α -trimming smoothing technique, which allows trimming these outliers. The α -smoothed predictor is defined as follows.

Definition 2 (α -smoothing [30]). *Let $h_{\mathbf{w}}$ be a stochastic predictor with $\mathcal{X}, \mathcal{Y}$, being its input and output spaces and $x \in \mathcal{X}$. Given $N \in \mathbb{N}_{>0}$ i.i.d. samples $\{\varepsilon_1, \dots, \varepsilon_N\}$ from a distribution $\mathbb{P}_{\varepsilon} \in \mathcal{P}(\mathcal{X} \ominus \{x\})$, define the random vector $\mathcal{H}(x) = (h_{\mathbf{w}}(x + \varepsilon_i))_{i \in [N]}$. Then, for $\alpha \in [0, \frac{1}{2})$, the α -smoothed predictor is defined as*

$$\tilde{\mathcal{H}}_{N,\alpha}(x) = \frac{1}{N - 2\lfloor \alpha N \rfloor} \sum_{i=1+\lfloor \alpha N \rfloor}^{N-\lfloor \alpha N \rfloor} \mathcal{H}(x)_{(i)}, \quad (1)$$

where the i -th order statistic in $\mathcal{H}(x)_{(i)}$ is taken element-wise.

α -smoothing allows tighter control over the probability that $\tilde{\mathcal{H}}_{N,\alpha}(x') \in \mathcal{R}$ for any convex set $\mathcal{R} \subseteq \mathcal{Y}$. However, when the predictor distribution is multi-modal, the above-mentioned technique inadvertently collapses its modes (Section 3). To solve this issue, we propose *clustered α -smoothing*. Before introducing clustered α -smoothing, we should note that clustering has previously been applied successfully in randomized smoothing to robustify large language models [39, 42] under the justification that adversarial examples are often located in small clusters adjacent to the dominant mode. However, unlike these previous works, the safety-critical settings considered in this manuscript require that we account for behavior of the predictor in those minor clusters and thus cannot simply prune them, as done in [39, 42].

3 Clustered α -Smoothing

In this section, we introduce a novel randomized smoothing method tailored to stochastic predictors, which we call *clustered α -smoothing*. The key idea is to (i) cluster samples in the output space, (ii) apply smoothing locally within each cluster, and (iii) combine the resulting predictors into a mixture distribution, with weights proportional to the size of each cluster. By combining the locally-smoothed predictors as a mixture, as illustrated in Figure 2, we prevent the mode collapse observed in α -smoothing [30]. To formally define clustered α -smoothing, we let $\mathcal{V} = \{\mathcal{V}_1, \dots, \mathcal{V}_M\}$ be a partition of the output space $\mathcal{Y}$ into M disjoint sets, each corresponding to a mode of the prediction. For now, we assume that $\mathcal{V}$ is given; in Section 4, we describe how to construct $\mathcal{V}$ via clustering so that it aligns with the modes of the predictor.

Definition 3 (Clustered α -smoothing). *Let $h_{\mathbf{w}}$ be a stochastic predictor, $x \in \mathcal{X}$ a point in its domain, and $\mathcal{V} = \{\mathcal{V}_1, \dots, \mathcal{V}_M\}$ a partition of $\mathcal{Y}$. Given $N \in \mathbb{N}_{>0}$ i.i.d. samples $\{\varepsilon_1, \dots, \varepsilon_N\}$ from a distribution $\mathbb{P}_{\varepsilon} \in \mathcal{P}(\mathcal{X} \ominus \{x\})$, we define the set of indexes $I_m \subset [N]$ as $I_m = \{i \in [N] :$*

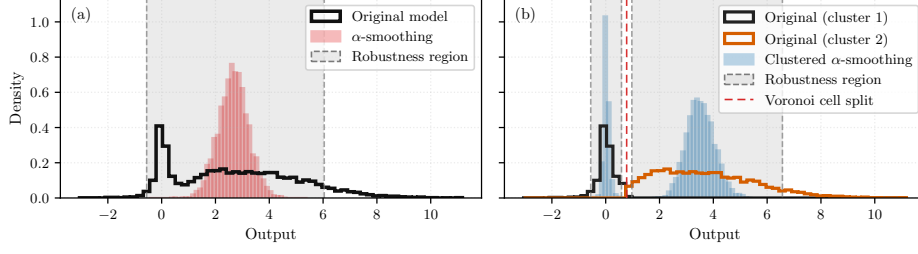


Figure 2: An example of α -smoothing and clustered α -smoothing at $x = 2$ of a stochastic base predictor $h_{\mathbf{w}}(x) = \mathbf{w}x$ where $\mathbf{w} \sim \sum_{i=1}^3 \pi_i \mathcal{N}(\cdot | \mu_i, \sigma_i^2)$ is a mixture of three Gaussians with means $\mu = (1, 0, 2)$ and equal variance $\sigma_i^2 = 0.01$, and weights $\pi = (0.2, 0.2, 0.6)$. The histograms are obtained by Monte Carlo sampling the distributions of $h_{\mathbf{w}}(x + \varepsilon)$, $\tilde{\mathcal{H}}_{N,\alpha,\{\mathcal{V}\}}(x)$ (in (a), equivalent to α -smoothing, see Remark 1), and our clustered α -smoothing technique, $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}}(x)$, in (b) constructed by Algorithm 2. The Voronoi cell split represents the partition $\mathcal{V} = \{\mathcal{V}_1, \mathcal{V}_2\}$ into two clusters, and the robustness regions in (b) denote sets that $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}}(x)$ belongs to with high probability (see Algorithm 1).

$h_{\mathbf{w}}(x + \varepsilon_i) \in \mathcal{V}_m\}$. Then, define $\mathcal{H}_{\mathcal{V}_m}(x) = (h_{\mathbf{w}}(x + \varepsilon_i))_{i \in I_m}$ and, for $\alpha \in [0, \frac{1}{2}]$ call

$$\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}(x) = \frac{1}{|I_m| - 2\lfloor \alpha |I_m| \rfloor} \sum_{i=1+\lfloor \alpha |I_m| \rfloor}^{|I_m| - \lfloor \alpha |I_m| \rfloor} \mathcal{H}_{\mathcal{V}_m}(x)_{(i)}, \quad (2)$$

where the i -th ordering statistic in $\mathcal{H}_{\mathcal{V}_m}(x)_{(i)}$ is taken element-wise. Then, the clustered α -smoothing is defined as follows

$$\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}}(x) = \sum_{m=1}^M \mathbf{1}_{\mathbf{z}=\mathbf{m}} \tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}(x) \quad (3)$$

where $\mathbf{z}$ is a categorical random variable with class weights $\pi^{(m)} = \frac{|I_m|}{N}$.

Intuitively, the set of indexes I_m identifies to which partition cell each prediction sample $h_{\mathbf{w}}(x + \varepsilon_i)$ belongs. Then $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}$ applies the α -trimming average to each partition cell individually. Finally, the clustered α -smoother $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}}(x)$ is constructed by randomly selecting each of the $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}$ with a probability proportional to how many samples are in each $\mathcal{V}_m$. The parameter α in Definition 3 refers to trimming a 2α fraction of samples, i.e., treating them as outliers, *per cluster*. When $\alpha = 0$, $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}(x)$ reduces to the mean smoothing, $\tilde{\mathcal{H}}_{N,0,\mathcal{V}_m}(x) = \frac{1}{|I_m|} \sum_{i \in I_m} h_{\mathbf{w}}(x + \varepsilon_i)$. When $\alpha \rightarrow \frac{1}{2}$, $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}(x)$ is equivalent to the median-smoothing, $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}(x) = \text{median}_{i \in I_m} (h_{\mathbf{w}}(x + \varepsilon_i))$.

Remark 1 (Relation with α -smoothing in [30]). When $\mathcal{V} = \{\mathcal{V}\} = \{\mathcal{Y}\}$, then Definition 3 is equivalent to the α -smoothing in (1). However, when distributions exhibit multi-modality or are not strongly symmetric, then the flexibility of our approach can lead to a substantial improvement of performance over [30], as we will show in Section 5.

Robustness Guarantees. A core advantage of randomized smoothing is the ability to guarantee that, for sufficiently small perturbations, the output remains within a desired set with high probability. In this subsection, we develop this type of guarantee for clustered α -smoothing. Our approach, which results in Theorem 3.1 below, is to apply the Neyman-Pearson Lemma [8] to each cluster/mode, which are then combined to obtain worst-case bounds for the smoothed predictor via a union-bound argument combined with correction to the fact that $\mathcal{V}$ is a partition of $\mathcal{X}$. To apply the lemma, we follow the predominant convention in the randomized smoothing literature and adopt $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$ for $\sigma^2 > 0$, which requires $\mathcal{X} = \mathbb{R}^d$.

Theorem 3.1 (Robustness certification for clustered α -smoothing). *Let $\mathcal{V} = \{\mathcal{V}_1, \dots, \mathcal{V}_M\}$ be a partition of the output space $\mathcal{Y}$. Assume that $\check{p}_{\mathcal{V}_m} \leq \mathbb{P}_{(\mathbf{w}, \varepsilon)}(h_{\mathbf{w}}(x + \varepsilon) \in \mathcal{V}_m) \leq \hat{p}_{\mathcal{V}_m}$ for each $\mathcal{V}_m$. Furthermore, let $\mathcal{L} \subset [M]$ and define $\tilde{\mathcal{R}} = \cup_{l \in \mathcal{L}} \mathcal{R}_l$ for convex sets $\mathcal{R}_l \subset \mathcal{V}_l$. Assume that $\mathbb{P}_{(\mathbf{w}, \varepsilon)}(h_{\mathbf{w}}(x + \varepsilon) \in \mathcal{R}_l) \geq \check{p}_{\mathcal{R}_l}$ for each $\mathcal{R}_l$. For $r \geq 0$, define for each $m \in [M]$ and $l \in \mathcal{L}$*

$$\underline{p}_{\mathcal{V}_m} = \Phi\left(\Phi^{-1}(\check{p}_{\mathcal{V}_m}) - \frac{r}{\sigma}\right), \quad \bar{p}_{\mathcal{V}_m} = \Phi\left(\Phi^{-1}(\hat{p}_{\mathcal{V}_m}) + \frac{r}{\sigma}\right), \quad \text{and} \quad \underline{p}_{\mathcal{R}_l} = \Phi\left(\Phi^{-1}(\check{p}_{\mathcal{R}_l}) - \frac{r}{\sigma}\right). \quad (4)$$

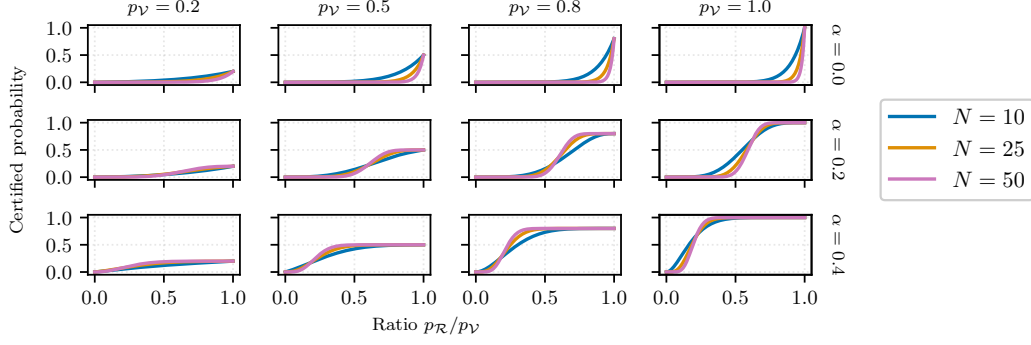


Figure 3: Probability bounds $\sum_{s=1}^N \frac{s}{N} \sum_{j=s-\lfloor \alpha s \rfloor}^s f_{s,j} \left(\frac{p_{\mathcal{R}}}{p_{\mathcal{V}}} \right) f_{N,s}(p_{\mathcal{V}})$ from (6) attributable to each mode for different combinations of number of smoothing samples N , trimming parameter α , partition set probability $p_{\mathcal{V}}$ and coverage ratio $p_{\mathcal{R}}/p_{\mathcal{V}}$. The right-most column (where $p_{\mathcal{V}} = 1$) is equivalent to the bound in [30], by noting that only the term $s = N$ in (6) is non-zero when $p_{\mathcal{V}} = p_{\mathcal{V}} = \bar{p}_{\mathcal{V}} = 1$.

Then, for any $\delta \in \mathbb{R}^d$ such that $\|\delta\|_2 \leq r$, it holds that

$$\begin{aligned} & \mathbb{P}_{(\mathbf{w}, \boldsymbol{\varepsilon}_1, \dots, \boldsymbol{\varepsilon}_N)} \left(\tilde{\mathcal{H}}_{N, \alpha, \mathbf{V}}(x + \delta) \in \tilde{\mathcal{R}} \right) \\ & \geq \inf_{\substack{p_{\mathcal{V}_m} \in [\underline{p}_{\mathcal{V}_m}, \bar{p}_{\mathcal{V}_m}] \\ \sum_{m=1}^M p_{\mathcal{V}_m} = 1}} \sum_{l \in \mathcal{L}} \sum_{s=1}^N \frac{s}{N} \sum_{j=s-\lfloor \alpha s \rfloor}^s f_{s,j} \left(\frac{p_{\mathcal{R}_l}}{p_{\mathcal{V}_l}} \right) f_{N,s}(p_{\mathcal{V}_l}). \end{aligned} \quad (5)$$

The proof of Theorem 3.1 can be found in Appendix A. For each partition region $\mathcal{V}_m$, the interval $[\underline{p}_{\mathcal{V}_m}, \bar{p}_{\mathcal{V}_m}]$ bounds the probability that $h_{\mathbf{w}}(x + \delta + \boldsymbol{\varepsilon}) \in \mathcal{V}_m$ for any perturbation δ with norm $\|\delta\|_2 \leq r$. Analogously, $\underline{p}_{\mathcal{R}_l}$ represents a probability lower bounds on the event $h_{\mathbf{w}}(x + \delta + \boldsymbol{\varepsilon}) \in \mathcal{R}_l$. Consequently, Theorem 3.1 combines these bounds to obtain a lower bound for the probability that $\tilde{\mathcal{H}}_{N, \alpha, \mathbf{V}}(x + \delta) \in \tilde{\mathcal{R}}$. Figure 3 shows how the various parameters affect the resulting bounds. It can be observed that a higher trimming parameter α leads to a significantly faster increase of the bound, as outlier output values are eliminated from the average predictor in (2), while higher ratios $\frac{p_{\mathcal{R}_l}}{p_{\mathcal{V}_l}}$ – i.e. larger coverage in the partition set – naturally increases the bound.

Before showing how to compute (5) in practice, it is instructive to examine the properties of the case $|\mathcal{L}| = 1$ first. In particular, the objective function in this case is independent from all other indices $[M] \setminus \mathcal{L}$, and so, as we formally prove in Lemma A.1, the bound in (5) simply reduces to

$$\begin{aligned} & \mathbb{P}_{(\mathbf{w}, \boldsymbol{\varepsilon}_1, \dots, \boldsymbol{\varepsilon}_N)} \left(\tilde{\mathcal{H}}_{N, \alpha, \mathbf{V}}(x + \delta) \in \mathcal{R}_l \right) \\ & \geq \sum_{s=1}^N \frac{s}{N} \sum_{j=s-\lfloor \alpha s \rfloor}^s f_{s,j} \left(\frac{p_{\mathcal{R}_l}}{\bar{p}_{\mathcal{V}_l}} \right) \min \left\{ f_{N,s} \left(\underline{p}_{\mathcal{V}_l} \right), f_{N,s} \left(\bar{p}_{\mathcal{V}_l} \right) \right\}. \end{aligned} \quad (6)$$

Solving the general case. Unfortunately, the computational convenience of the case $|\mathcal{L}| = 1$ does not extend to the general case $|\mathcal{L}| > 1$, as in this setting (5) consists of a non-convex optimization problem. Instead, to achieve a computationally tractable result, we show how the optimization problem in (5) can be lower bounded by a linear program via *anchor points* and an error that can be controlled by the user. This is presented in Proposition 3.2.

Proposition 3.2. Assume that $\underline{p}_{\mathcal{V}_m}, \bar{p}_{\mathcal{V}_m}$ for every $m \in [M]$ and $\underline{p}_{\mathcal{R}_l}$ for every $l \in \mathcal{L}$ are given. Define for each component $l \in \mathcal{L}$ the anchor points $p_{\mathcal{V}_l}^{(1)} < p_{\mathcal{V}_l}^{(2)} < \dots < p_{\mathcal{V}_l}^{(K)}, p_{\mathcal{V}_l}^{(k)} \in [\underline{p}_{\mathcal{V}_l}, \bar{p}_{\mathcal{V}_l}]$.

Then, it holds that

$$\inf_{\substack{p_{\mathcal{V}_m} \in [\underline{p}_{\mathcal{V}_m}, \bar{p}_{\mathcal{V}_m}] \\ \sum_{m=1}^M p_{\mathcal{V}_m} = 1}} \sum_{l \in \mathcal{L}} \sum_{s=1}^N \frac{s}{N} \sum_{j=s-\lfloor \alpha s \rfloor}^s f_{s,j} \left(\frac{\underline{p}_{\mathcal{R}_l}}{p_{\mathcal{V}_l}} \right) f_{N,s}(p_{\mathcal{V}_l}) \quad (7)$$

$$\geq \inf_{\mathbf{p}, \boldsymbol{\lambda}_1, \dots, \boldsymbol{\lambda}_{|\mathcal{L}|}} \sum_{l \in \mathcal{L}} \sum_{k=1}^K \lambda_l^{(k)} g_l(p_{\mathcal{V}_l}^{(k)}) - \sum_{l \in \mathcal{L}} \left[\frac{N}{1 - \bar{p}_{\mathcal{V}_l}} + \frac{N}{\underline{p}_{\mathcal{V}_l} - \underline{p}_{\mathcal{R}_l}} \right] h_{l,K} \quad (8)$$

$$\text{s.t. } p_{\mathcal{V}_m} \in [\underline{p}_{\mathcal{V}_m}, \bar{p}_{\mathcal{V}_m}], \sum_{m=1}^M p_{\mathcal{V}_m} = 1, \sum_{k=1}^K \lambda_l^{(k)} p_{\mathcal{V}_l}^{(k)} = p_{\mathcal{V}_l}, \boldsymbol{\lambda}_l \in \Delta_K, \quad (9)$$

where the variable $h_{l,K}$ and function g_l are given by $h_{l,K} = \max_{p \in [\underline{p}_{\mathcal{V}_l}, \bar{p}_{\mathcal{V}_l}]} \min_{k \in [K]} |p - p_{\mathcal{V}_l}^{(k)}|$ and $g_l(p) = \sum_{s=1}^N \frac{s}{N} f_{N,s}(p) \sum_{j=s-\lfloor \alpha s \rfloor}^s f_{s,j} \left(\frac{\underline{p}_{\mathcal{R}_l}}{p} \right)$.

The intuition behind Proposition 3.2 is the following. We seek a tractable lower bound on the infimum in (7). This is generally intractable. Consequently, instead, we replace $p_{\mathcal{R}_l}$ by $\underline{p}_{\mathcal{R}_l}$ to obtain a valid lower bound, and then approximate the convex envelope of each inner component using a finite set of evaluation points – the *anchor points* – leading to a linear program in which the coupling constraint is handled natively. The bound is asymptotically exact as the number of anchor points K increases, at the cost of increasing the size of the linear program. We call the second term of (8) the residual, which represents an adjustment needed due to the fact that we only approximate the convex envelope. Note that, as we show in Appendix G, one can always achieve an arbitrarily small residual value by increasing K .

Remark 2 (Extension to L_ρ -norm attack). Given $\rho \in [2, \infty) \cup \{\infty\}$, the guarantee (5) also holds for $\delta \in \mathbb{R}^d$ such that $\|\delta\|_\rho \leq d^{\frac{1}{\rho} - \frac{1}{2}} r$ by a straight-forward application of Holder's inequality [34].

4 Algorithm

In Theorem 3.1, given a point $x \in \mathcal{X}$, we need to compute probability bounds for the events $h_{\mathbf{w}}(x + \varepsilon) \in \mathcal{V}_m$ and $h_{\mathbf{w}}(x + \varepsilon) \in \mathcal{R}_l$, where $\mathcal{V} = \{\mathcal{V}_m\}_{m=1}^M$ is a partition of the output space $\mathcal{Y}$ into M sets, and $\mathcal{R}_l \subseteq \mathcal{V}_l$ are given subsets. Given the (possibly) non-linearity of the underlying deterministic function h_w on both the parameters w and input x , computing those quantities explicitly is generally intractable. Thus, one needs to resort to sample-based methods, for which probability bounds are estimated with high-confidence.

Proposition 4.1 (High-confidence probability bounds for sets). Let $\mathcal{V} = \{\mathcal{V}_m\}_{m=1}^M$ be a partition of the space $\mathcal{X}$, and $\mathcal{R}_l \subseteq \mathcal{V}_l$ be given subsets. Given a confidence parameter $\beta \in [0, 1]$, and a number of samples $\bar{N} \in \mathbb{N}_{>0}$, let $Z_{\mathcal{V}_m} = \sum_{i=1}^{\bar{N}} \mathbf{1}_{h_{w_i}(x + \varepsilon_i) \in \mathcal{V}_m}$ and $Z_{\mathcal{R}_m} = \sum_{i=1}^{\bar{N}} \mathbf{1}_{h_{w_i}(x + \varepsilon_i) \in \mathcal{R}_m}$, where (w_i, ε_i) are i.i.d. samples from $\mathbb{P}_{(\mathbf{w}, \varepsilon)}$. If $\hat{p}_{\mathcal{V}_m}, \hat{p}_{\mathcal{R}_m} \in [0, 1]$ are such that¹

$$\frac{\beta}{3M} = \sum_{j=Z_{\mathcal{V}_m}}^{\bar{N}} f_{\bar{N},j}(\hat{p}_{\mathcal{V}_m}), \quad \frac{\beta}{3M} = \sum_{j=0}^{Z_{\mathcal{V}_m}} f_{\bar{N},j}(\hat{p}_{\mathcal{V}_m}), \quad \text{and} \quad \frac{\beta}{3M} = \sum_{j=Z_{\mathcal{R}_m}}^{\bar{N}} f_{\bar{N},j}(\hat{p}_{\mathcal{R}_m}), \quad (10)$$

then, the assumptions in Theorem 3.1 hold with confidence $1 - \beta$.

Proposition 4.1 comes from the Clopper-Pearson Lemma [8] combined with a union-bound argument. It gives a natural way to construct the partition $\mathcal{V}$ and sets $\mathcal{R}$: after observing the samples $\{h_{w_i}(x + \varepsilon_i)\}$, one may place the sets $\mathcal{V}_m$ and $\mathcal{R}_m$ so that the number of outputs inside them (approximately) induces the desired probability level. This reasoning is the core principle of Algorithm 1.

Given the user's desired coverage level p , Algorithm 1 generates the sets $\mathcal{R}_m$ to cover a fraction p of the samples within $\mathcal{V}_m$ (Line 9). This parameter can be used to calibrate the trade-off between the size of the coverage sets and their certified probability. Next, by choosing the partition of the input

¹If $Z_{\mathcal{V}_m} = 0$ (resp. $Z_{\mathcal{R}_m} = 0$), then $\hat{p}_{\mathcal{V}_m} = 0$ (resp. $\hat{p}_{\mathcal{R}_m} = 0$). Alternatively, if $Z_{\mathcal{V}_m} = \bar{N}$, then $\hat{p}_{\mathcal{V}_m} = 1$.

Algorithm 1 Construction of partition $\mathcal{V} = \{\mathcal{V}_1, \dots, \mathcal{V}_M\}$ and sets $\mathcal{R} = \{\mathcal{R}_1, \dots, \mathcal{R}_M\}$ from samples such that $\mathcal{R}_i \subset \mathcal{V}_i$ and $\mathbb{P}(h_{\mathbf{w}}(x + \varepsilon) \in \mathcal{R}_i) / \mathbb{P}(h_{\mathbf{w}}(x + \varepsilon) \in \mathcal{V}_i) \approx p$.

```

1: function CONSTRUCTCOVERAGESETS( $x, h_{\mathbf{w}}, p, N$ )
2:   Outputs  $\leftarrow \emptyset$ 
3:   for  $i \leftarrow 1, \dots, N$  do
4:     Draw  $\varepsilon_i \sim \mathcal{N}(0, \sigma^2 I)$  and  $w_i \sim \mathbb{P}_{\mathbf{w}}$ 
5:     Compute  $h_{w_i}(x + \varepsilon_i)$  and append to Outputs
6:   Cluster the elements of Outputs  $\triangleright$  e.g., using DBSCAN, DP-means, UMAP.
7:   Denote the resulting clusters by  $C_1, \dots, C_M$ 
8:   for  $m \leftarrow 1, \dots, M$  do
9:      $C_m^{(p)} \leftarrow$  Select approximately a fraction  $p$  of the points in  $C_m$ 
10:     $\mathcal{R}_m \leftarrow \text{ENVELOPE}(C_m^{(p)})$   $\triangleright$  e.g., convex hull, hyperrectangle.
11:   $\mathcal{V} \leftarrow$  generalized Voronoi partition associated with  $\mathcal{R}$ 
12:  return partition  $\mathcal{V}$  and coverage regions  $\mathcal{R}$ 

```

space as a generalized Voronoi partition (closest *region* instead of point, Line 11; see Appendix E for details) [25], we enforce that the regions are contained in their partition, which is a requirement for Theorem 3.1. Although computing the (generalized) Voronoi partition associated with a set of regions $\mathcal{R}$ is hard (see Algorithm 2), we only need to check whether a point belongs to $\mathcal{V}_m$, which is equivalent to checking if $\mathcal{R}_m$ is the closest region to this point. Thus, in practice, $\mathcal{V}$ is implicitly defined by $\mathcal{R}$. With partition $\mathcal{V}$, we can compute the robustness bounds introduced in Theorem 3.1 for the clustered α -smoother $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}}$, as presented in Algorithm 2. Appendix C expands on the parameters of Algorithm 2 and their impact on the coverage regions and certified probabilities.

Algorithm 2 Robustness certification of $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}}$ at $x \in \mathcal{X}^2$.

Require: Point $x \in \mathbb{R}^d$, stochastic function $h_{\mathbf{w}}$, smoothing parameters $\alpha \in [0, 1]$ and $N \in \mathbb{N}_{>0}$, coverage level $p \in [0, 1]$, robustness radius $r \geq 0$, number of samples $\bar{N} \in \mathbb{N}_{>0}$, and confidence level $\beta \in (0, 1)$.

Ensure: With confidence $1 - \beta$, for any $m \in [M]$, $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}}(x + \delta) \in \tilde{\mathcal{R}}$ with probability given by (5) for any $\delta \in \mathbb{R}^d$ such that $\|\delta\|_2 \leq r$.

```

1:  $\mathcal{V}, \mathcal{R} \leftarrow \text{CONSTRUCTCOVERAGESETS}(x, h_{\mathbf{w}}, p, \bar{N})$ 
2: Construct  $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}}(x)$ 
3: Compute  $\check{p}_{\mathcal{V}_m}, \hat{p}_{\mathcal{V}_m}, \check{p}_{\mathcal{R}_m} \in [0, 1]$  with Proposition 4.1 for  $m \in [M]$  (using  $\beta$  and  $\bar{N}$ )
4: Compute  $\underline{p}_{\mathcal{V}_m}, \bar{p}_{\mathcal{V}_m}, \underline{p}_{\mathcal{R}_m}$  using (4) for  $m \in [M]$ 
5: return  $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}}(x)$  and (5) certificate.

```

The time complexity of Algorithm 2 is generally dominated by the repeated inference cost of the base classifier. It is thus governed by the number of samples required, which is determined by (i) the data patterns and clustering algorithm, (ii) the confidence bound (binomial inference, see Proposition 4.1), and (iii) the probability bound in Theorem 3.1.

5 Experiments

We evaluated our framework on two benchmarks: stochastic trajectory prediction and multi-modal RL quadrotor control. For both experiments, we use the minimum bounding hyperrectangle for the envelope of each cluster (Appendix D provides further details on the experiment setup)³. All experiments were run on an Intel Core i7-1365U CPU with 16GB of RAM.

Trajectory Prediction. We consider the stochastic *TrajFlow* prediction model [26], a normalizing flow-based model (details in Appendix D.1), and apply it to the *L-GAP* driving simulator dataset [47].

²The quality of the smoothed predictor $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}}(x)$ depends critically on the choice of partition $\mathcal{V}$. In Algorithm 1, we construct $\mathcal{V}$ locally around x to ensure good performance in its neighborhood. However, to preserve the validity of Theorem 3.1, the same partition must be used for all points x' within a δ -neighborhood of x .

³The code is publicly available at <https://github.com/EduardoFMDCosta/ClusteredRandomizedSmoothing>.

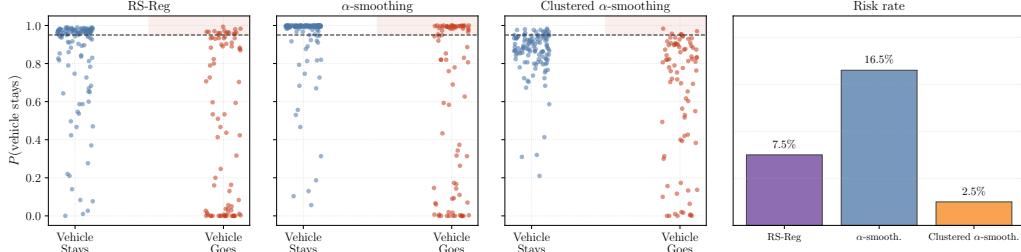


Figure 4: Safety analysis for 200 randomly selected inputs x_i from the L-GAP dataset.

In this dataset, human drivers face a decision whether to turn left in front of an oncoming vehicle (*go*) or wait for it to pass before turning (*yield*). The recorded trajectories are converted into control inputs (acceleration and curvature) using the unicycle model [37]. The resulting stochastic predictor is then defined as $h_w : \mathbb{R}^{18} \rightarrow \mathbb{R}^{40}$, where the input space $\mathbb{R}^{18}$ encodes the two control actions that underlie the 9 transitions between the 10 past recorded positions of the target agent. The output space $\mathbb{R}^{40}$ corresponds to the position of the target vehicle over 20 time-steps, here corresponding to a horizon of 2 s.

We start with a safety analysis comparing our approach to α -smoothing and RS-Reg [31] (which is equivalent to smoothing without any trimming, i.e. $\alpha = 0$ in (1)) using the L-GAP dataset. In particular, we evaluate the predicted probability that the vehicle will adopt the behavior *stay*⁴ and we make the ego vehicle *go* if it predicts a *stay* probability for the other higher than a decision threshold of 95%. We then use the actual trajectory of the predicted vehicle to assess the risk of this decision: if both vehicles decided to *go*, we classify it as a *risky decision*. We then report the risk rate for 200 inputs in our database. We show that RS-Reg has a risk rate of 7.5%, α -smoothing of 16.5%, while ours only 2.5%. The main mechanism behind this result is that while α -smoothing and RS-Reg are often highly sure of the oncoming vehicle’s behavior (due to the mode collapse for both RS-Reg and α -smoothing), our clustered smoother allows for behavioral uncertainty that makes it more cautious about adopting the *go* decision, as illustrated in Figure 4.

Second, for a quantitative comparison of the discrepancy between the output distributions from each approach, we select a random subset of 200 input trajectories x_i from L-GAP and compute a sample approximation of the 2-Wasserstein distance $\mathbb{W}_2$ between the noisy predictor $h_w(x_i + \epsilon)$ and α -smoothing, RS-Reg, and ours — note that we are thus comparing distributions embedded in a 40-dimensional space. We found that our clustered α -smoothing reduces the mean 2-Wasserstein distance to $h_w(x + \epsilon)$ in this subset of data in 27% compared to α -smoothing (3.80 vs. 5.18; Table 1). This confirms the intuition that the distributional flexibility of our method generally yields predictions that are closer to the original predictor, thereby better capturing its uncertainty (as also illustrated in Figure 7 in Appendix B).

Method	Wasserstein $\mathbb{W}_2$	
	Trajectory ($\mathbb{R}^{40}$)	Last step ($\mathbb{R}^2$)
Ours	3.80 $\pm$ 1.59	1.49 $\pm$ 0.63
RS-Reg	4.70 $\pm$ 2.09	1.84 $\pm$ 0.84
α -smoothing	5.18 $\pm$ 2.19	2.04 $\pm$ 0.84

Table 1: Comparison between smoothers and $h_w(x_i + \epsilon)$ in terms of $\mathbb{W}_2$ for 200 inputs x_i .

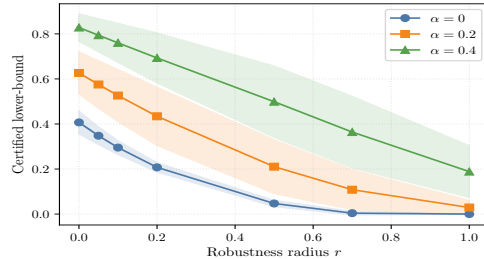


Figure 5: Robustness lower-bounds (with one std. dev. shaded band) for 20 distinct inputs x_i .

Finally, we perform a parameter analysis of the robustness lower-bounds in Theorem 3.1 for various trimming parameters $\alpha \in \{0.0, 0.2, 0.4\}$ and robustness radii $r \geq 0$, where the sets $\mathcal{R}$ are constructed according to Algorithm 1 enveloping a fraction $p = 0.85$, for different inputs in our database. In Figure 5, we observe that certified lower bounds are higher when the robustness radius is smaller, as this allows smaller perturbations from the reference input, and when the trimming parameter α is higher (so that outliers are filtered out, so we can be more certain about where the smoother

⁴The behaviors *stay* and *go* are defined in the Appendix D (*Behavioral clustering*).

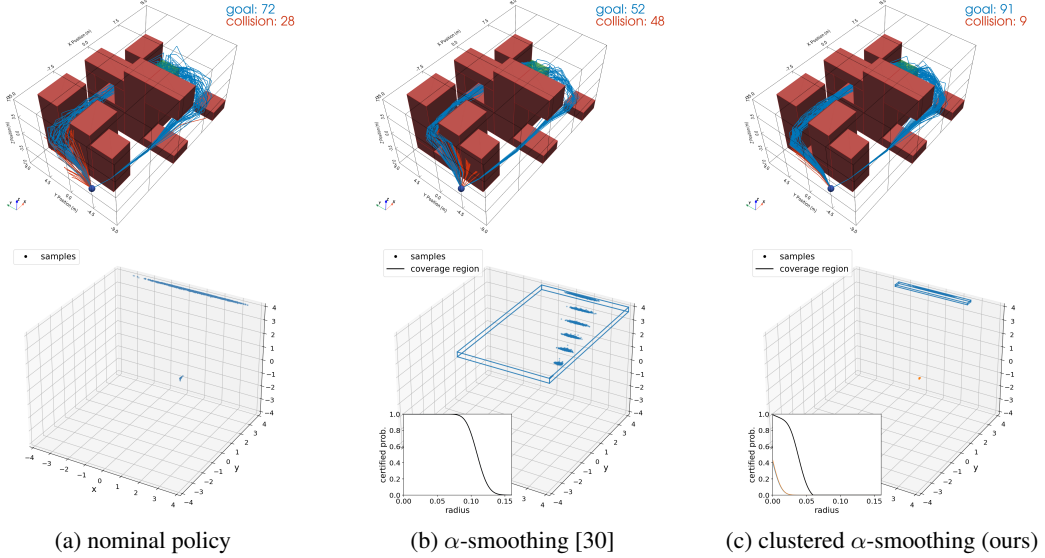


Figure 6: Top row: rollouts of the quadrotor from the starting state (blue marker) where the controller predicts a bi-modal action distribution, corresponding to navigating through two “holes” in the obstacles towards the goal region (green). Bottom row: action distributions at the initial state $x = (-3.5, 0, -0.35, 0, 0.7, 0)$. The smoothed predictors are constructed using 30 samples. Insets: the certified probability of robustness of the smoothed policies with respect to their coverage regions (boxes) as a function of perturbation radius. The quantization in (b) arises from cross-mode smoothing with a limited number of samples (30 samples, α -trimmed to 6) (analyzed further in Appendix F).

probability mass is concentrated). Despite the solid certification suggested by Figure 5, decreasing the conservativeness of the lower bounds in Theorem 3.1 constitutes an interesting research direction.

Multi-modal quadrotor control. Quadrotors are safety-critical systems where robustness directly affects deployability. We consider a navigation task in which the quadrotor must reach a goal while avoiding numerous obstacles (see Figure 6), under stochastic dynamics with full state observability [3]. Details of benchmark can be found in Appendix D.2. The task is inherently multi-modal: obstacles between the quadrotor and its goal admit passage on either side, and wind can shift probability mass between such modes. The controller is an RL policy that outputs a Gaussian mixture over actions with predicted component means and mixture weights, and fixed variances [32]. The prediction of a mixture enables navigation around obstacles through either path and rejection of wind-induced disturbances. This is precisely the setting where randomized and α -smoothing fails: perturbations are averaged such that different modes are collapsed to their (weighted) mean, which often is to navigate directly into an obstacle. Clustered smoothing recovers meaningful guarantees by treating each cluster individually.

Visual inspection of policies (Figure 6) shows that the nominal policy is bi-modal, corresponding to navigating through one of two paths through the obstacles to the goal. However, the α -smoothing destroys the multi-modality and ultimately predicts the (trimmed) average, which corresponds to navigating directly at the obstacles. Instead, our framework preserves the modes and only tightens the clusters. In the simulation shown in Figure 6, the nominal policy reaches the goal in 72 of the 100 trajectories sampled and crashes in 28 trajectories. The α -smoothed policy crashes in 48 trajectories, while our clustered α -smoothed policy only crashes in 9 trajectories. The certified probability as a function of perturbation radius in Figure 6 suggests that our method is penalized more for larger input radii r , by the correction of $\underline{p}_{V_m}, \bar{p}_{V_m}$ in (4).

Note that the guarantees of the smoothed predictors are with respect to the controller output at each time step, while the benchmark operates in a receding horizon fashion. Nonetheless, the empirical results show significant benefit in applying our method compared to both the base policy and regular α -smoothing. Moreover, our approach is agnostic to the underlying predictor and is thus applicable to controllers such as MPC and motion planners such as RRT, both of which are important methods in robotic applications.

6 Limitations

Randomized smoothing relies on repeated inference. In certain contexts (e.g. large neural networks), this may be prohibitive, in particular when the base predictor has a non-negligible computational cost. That our method is agnostic to the clustering algorithm is both a strength and a weakness: the quality of the result depends strongly on the choice, and thus, one will need to assess the clustering to choose a suitable algorithm. Theory-wise, we identify four primary limitations. First, defining $\mathcal{R}$ as a union of disjoint and convex sets is a key component to enabling the guarantees, but this is restrictive, e.g., if the clusters cannot be cleanly separated as convex sets such as the famous half-moon toy example. The second limitation – that certification is with respect to the smoothed predictor – is key, as one cannot apply our method to robustify the prediction if parity with the base regressor is crucial, such as physics and dynamical systems *simulations*. Third, in (4), if one wants to reduce the noise but maintain the probability bound, it is necessary to fix r/σ , and thus, r must be proportional to the reciprocal of σ . Fourth, as we have shown empirically, the lower bounds from Theorem 3.1 can become more conservative with the inclusion of new clusters. Consequently, either a lower certified radius or a larger noise level may be needed to guarantee robustness in highly multi-modal settings.

7 Conclusion

In this work, we addressed a key limitation of randomized smoothing for regression: the collapse of multi-modal predictions under averaging. We proposed *clustered α -smoothing*, which preserves multi-modality by applying α -trimmed smoothing within clusters and combining the results as a mixture. We established probabilistic robustness guarantees and demonstrated significant empirical gains, reducing the Wasserstein distance by 27% in trajectory prediction and the collision rate by 81% in quadrotor control. Several directions remain for future work. In particular, our current approach assumes a fixed partition of the output space $\mathcal{V}$. However, recent work in randomized smoothing [40, 2] suggests that input-dependent smoothing distributions can improve performance. Extending our framework to allow input-dependent smoothing and partitions $\mathcal{V}(x)$ is therefore a promising and challenging direction.

References

- [1] Steven Adams, Andrea Patane, Morteza Lahijanian, and Luca Laurenti. Bnn-dp: robustness certification of bayesian neural networks via dynamic programming. In *International Conference on Machine Learning*, pages 133–151. PMLR, 2023.
- [2] Motasem Alfarra, Adel Bibi, Philip HS Torr, and Bernard Ghanem. Data dependent randomized smoothing. In *Uncertainty in Artificial Intelligence*, pages 64–74. PMLR, 2022.
- [3] Thom Badings, Licio Romao, Alessandro Abate, David Parker, Hasan A Poonawala, Marielle Stoelinga, and Nils Jansen. Robust control for dynamical systems with non-gaussian noise via formal abstractions. *Journal of Artificial Intelligence Research*, 76:341–391, 2023.
- [4] Inhwon Bae, Young-Jae Park, and Hae-Gon Jeon. Singulartjectory: Universal trajectory predictor using diffusion model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17890–17901, 2024.
- [5] Holger Caesar, Varun Bankiti, Alex H. Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. nuScenes: A Multimodal Dataset for Autonomous Driving. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11618–11628. IEEE Computer Society, June 2020.
- [6] Yipu Chen, Haotian Xue, and Yongxin Chen. Diffusion policy attacker: Crafting adversarial attacks for diffusion-based policies. *Advances in Neural Information Processing Systems*, 37:119614–119637, 2024.
- [7] Ping-yeh Chiang, Michael Curry, Ahmed Abdelkader, Aounon Kumar, John Dickerson, and Tom Goldstein. Detection as regression: Certified object detection with median smoothing. *Advances in Neural Information Processing Systems*, 33:1275–1286, 2020.
- [8] Jeremy Cohen, Elan Rosenfeld, and Zico Kolter. Certified adversarial robustness via randomized smoothing. In *international conference on machine learning*, pages 1310–1320. PMLR, 2019.
- [9] Ruilong Deng, Gaoxi Xiao, Rongxing Lu, Hao Liang, and Athanasios V Vasilakos. False data injection on state estimation in power systems—attacks, impacts, and defense: A survey. *IEEE transactions on industrial informatics*, 13(2):411–423, 2016.

- [10] P Kingma Diederik and Welling Max. An introduction to variational autoencoders. *Foundations and Trends® in Machine Learning*, 12(4):307–392, 2019.
- [11] Hai Duong, ThanhVu Nguyen, and Matthew B Dwyer. Neursat: A high-performance verification tool for deep neural networks. In *International Conference on Computer Aided Verification*, pages 409–423. Springer, 2025.
- [12] Andrew Foong, David Burt, Yingzhen Li, and Richard Turner. On the expressiveness of approximate inference in bayesian neural networks. *Advances in Neural Information Processing Systems*, 33:15897–15908, 2020.
- [13] Timon Gehr, Matthew Mirman, Dana Drachler-Cohen, Petar Tsankov, Swarat Chaudhuri, and Martin Vechev. Ai2: Safety and robustness certification of neural networks with abstract interpretation. In *2018 IEEE symposium on security and privacy (SP)*, pages 3–18. IEEE, 2018.
- [14] Agathe Girard, Carl Rasmussen, Joaquin Q Candela, and Roderick Murray-Smith. Gaussian process priors with uncertain inputs application to multiple-step ahead time series forecasting. *Advances in neural information processing systems*, 15, 2002.
- [15] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.
- [16] Guy Katz, Clark Barrett, David L Dill, Kyle Julian, and Mykel J Kochenderfer. Reluplex: An efficient smt solver for verifying deep neural networks. In *International conference on computer aided verification*, pages 97–117. Springer, 2017.
- [17] Mathias Lecuyer, Vaggelis Atlidakis, Roxana Geambasu, Daniel Hsu, and Suman Jana. Certified robustness to adversarial examples with differential privacy. In *2019 IEEE symposium on security and privacy (SP)*, pages 656–672. IEEE, 2019.
- [18] Alexander Levine and Soheil Feizi. Wasserstein smoothing: Certified robustness against wasserstein adversarial attacks. In *International conference on artificial intelligence and statistics*, pages 3938–3947. PMLR, 2020.
- [19] Bai Li, Changyou Chen, Wenlin Wang, and Lawrence Carin. Certified adversarial robustness with additive noise. *Advances in neural information processing systems*, 32, 2019.
- [20] Xuanqing Liu, Minhao Cheng, Huan Zhang, and Cho-Jui Hsieh. Towards robust neural networks via random self-ensemble. In *Proceedings of the european conference on computer vision (ECCV)*, pages 369–385, 2018.
- [21] Yulong Lu and Jianfeng Lu. A universal approximation theorem of deep neural networks for expressing probability distributions. *Advances in neural information processing systems*, 33:3094–3105, 2020.
- [22] Aleksander Madry, Aleksandar Makelev, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. In *6th International Conference on Learning Representations, ICLR*, 2018.
- [23] Charles Meyers, Tommy Löfstedt, and Erik Elmroth. Safety-critical computer vision: an empirical survey of adversarial evasion attacks and defenses on computer vision systems. *Artificial Intelligence Review*, 56(Suppl 1):217–251, 2023.
- [24] Rhiannon Micheltore, Matthew Wicker, Luca Laurenti, Luca Cardelli, Yarin Gal, and Marta Kwiatkowska. Uncertainty quantification with statistical guarantees in end-to-end autonomous driving control. In *2020 IEEE international conference on robotics and automation (ICRA)*, pages 7344–7350. IEEE, 2020.
- [25] Victor Milenkovic. Robust construction of the voronoi diagram of a polyhedron. In *Canadian Conference on Computational Geometry*, volume 93, pages 473–478, 1993.
- [26] Anna Mészáros, Julian F. Schumann, Javier Alonso-Mora, Arkady Zgonnikov, and Jens Kober. TrajFlow: Learning Distributions over Trajectories for Human Behavior Prediction. In *2024 IEEE Intelligent Vehicles Symposium (IV)*, Jeju, June 2024.
- [27] Nigamaa Nayakanti, Rami Al-Rfou, Aurick Zhou, Kratarth Goel, Khaled S. Refaat, and Benjamin Sapp. Wayformer: Motion Forecasting via Simple & Efficient Attention Networks. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2980–2987, May 2023.
- [28] George Papamakarios, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing flows for probabilistic modeling and inference. *Journal of Machine Learning Research*, 22(57):1–64, 2021.
- [29] Zeyu Qin, Yanbo Fan, Yi Liu, Li Shen, Yong Zhang, Jue Wang, and Baoyuan Wu. Boosting the transferability of adversarial attacks with reverse adversarial perturbation. *Advances in neural information processing systems*, 35:29845–29858, 2022.
- [30] Aref Rekavandi, Farhad Farokhi, Olga Ohrimenko, and Benjamin Rubinstein. Certified adversarial robustness via randomized α -smoothing for regression models. *Advances in Neural Information Processing Systems*, 37:134127–134150, 2024.

- [31] Aref Miri Rekavandi, Olga Ohrimenko, and Benjamin IP Rubinstein. Rs-reg: Probabilistic and robust certified regression through randomized smoothing. *Transactions on Machine Learning Research*, 2025.
- [32] Jie Ren, Yewen Li, Zihan Ding, Wei Pan, and Hao Dong. Probabilistic mixture-of-experts for efficient deep reinforcement learning. *arXiv preprint arXiv:2104.09122*, 2021.
- [33] Andy Rivas, Gregory Kyriakos Delipei, and Jason Hou. Predictions of component remaining useful lifetime using bayesian neural network. *Progress in Nuclear Energy*, 146:104143, 2022.
- [34] Walter Rudin. Real and complex analysis, 1987.
- [35] Hadi Salman, Jerry Li, Ilya Razenshteyn, Pengchuan Zhang, Huan Zhang, Sebastien Bubeck, and Greg Yang. Provably robust deep learning via adversarially trained smoothed classifiers. *Advances in neural information processing systems*, 32, 2019.
- [36] Julian F Schumann, Eduardo Figueiredo, Frederik Baymler Mathiesen, Luca Laurenti, Jens Kober, and Arkady Zgonnikov. Evaluating randomized smoothing as a defense against adversarial attacks in trajectory prediction. *arXiv preprint arXiv:2603.10821*, 2026.
- [37] Julian F. Schumann, Jeroen Hagenus, Frederik Baymler Mathiesen, and Arkady Zgonnikov. Realistic Adversarial Attacks for Robustness Evaluation of Trajectory Prediction Models via Future State Perturbation. *ACM Journal on Autonomous Transportation Systems*, 2026.
- [38] Julian F Schumann, Anna Mészáros, Jens Kober, and Arkady Zgonnikov. STEP: Structured training and evaluation platform for benchmarking trajectory prediction models. *arXiv preprint arXiv:2509.14801*, 2025.
- [39] Guangzhi Su, Shuchang Huang, Yutong Ke, Zhuohang Liu, Long Qian, and Kaizhu Huang. Smoothguard: Defending multimodal large language models with noise perturbation and clustering aggregation. *arXiv preprint arXiv:2510.26830*, 2025.
- [40] Peter Sukenik, Aleksei Kuvshinov, and Stephan Günnemann. Intriguing properties of input-dependent randomized smoothing. In *International Conference on Machine Learning*, 2021.
- [41] Hoang-Dung Tran, Xiaodong Yang, Diego Manzananas Lopez, Patrick Musau, Luan Viet Nguyen, Weiming Xiang, Stanley Bak, and Taylor T Johnson. Nnv: the neural network verification tool for deep neural networks and learning-enabled cyber-physical systems. In *International conference on computer aided verification*, pages 3–17. Springer, 2020.
- [42] Zixia Wang, Gaojie Jin, Jia Hu, and Ronghui Mu. Clucert: Certifying llm robustness via clustering-guided denoising smoothing. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 37998–38006, 2026.
- [43] Anqi Wu, Sebastian Nowozin, Edward Meeds, Richard E. Turner, José Miguel Hernández-Lobato, and Alexander L. Gaunt. Deterministic variational inference for robust bayesian neural networks. In *7th International Conference on Learning Representations, ICLR*, 2019.
- [44] Haoze Wu, Omri Isac, Aleksandar Zeljić, Teruhiro Tagomori, Matthew Daggitt, Wen Kokke, Idan Refaeli, Guy Amir, Kyle Julian, Shahaf Bassan, et al. Marabou 2.0: a versatile formal analyzer of neural networks. In *International Conference on Computer Aided Verification*, pages 249–264. Springer, 2024.
- [45] Kaidi Xu, Zhouxing Shi, Huan Zhang, Yihan Wang, Kai-Wei Chang, Minlie Huang, Bhavya Kailkhura, Xue Lin, and Cho-Jui Hsieh. Automatic perturbation analysis for scalable certified robustness and beyond. *Advances in Neural Information Processing Systems*, 33:1129–1141, 2020.
- [46] Greg Yang, Tony Duan, J Edward Hu, Hadi Salman, Ilya Razenshteyn, and Jerry Li. Randomized smoothing of all shapes and sizes. In *International conference on machine learning*, pages 10693–10705. PMLR, 2020.
- [47] Arkady Zgonnikov, David Abbink, and Gustav Markkula. Should I Stay or Should I Go? Cognitive Modeling of Left-Turn Gap Acceptance Decisions in Human Drivers. *Human Factors: The Journal of the Human Factors and Ergonomics Society*, 66(5):1399–1413, May 2024.
- [48] Huan Zhang, Tsui-Wei Weng, Pin-Yu Chen, Cho-Jui Hsieh, and Luca Daniel. Efficient neural network robustness certification with general activation functions. *Advances in neural information processing systems*, 31, 2018.
- [49] Liangliang Zhuang, Ancha Xu, and Xiao-Lin Wang. A prognostic driven predictive maintenance framework based on bayesian deep learning. *Reliability Engineering & System Safety*, 234:109181, 2023.

A Technical Proofs

A.1 Proof of Theorem 3.1

As Theorem 3.1 is built from a union-bound argument over $\mathcal{R}_m$, we start by introducing the following Lemma A.1, which bounds the probability $\mathbb{P}_{(\mathbf{w}, \underline{\varepsilon}_1, \dots, \underline{\varepsilon}_N)} \left(\tilde{\mathcal{H}}_{N, \alpha, \mathbf{V}}(x + \delta) \in \mathcal{R}_m \right)$. For notational convenience, we write $\underline{\varepsilon}$ to mean $\varepsilon_1, \dots, \varepsilon_N$ for the remainder of this appendix.

Lemma A.1. *Let $\mathbf{V} = \{\mathcal{V}_1, \dots, \mathcal{V}_M\}$ be a partition of the space $\mathcal{X}$. Assume that $\check{p}_{\mathcal{V}_m} \leq \mathbb{P}_{(\mathbf{w}, \underline{\varepsilon})} (h_{\mathbf{w}}(x + \underline{\varepsilon}) \in \mathcal{V}_m) \leq \hat{p}_{\mathcal{V}_m}$ and $\mathbb{P}_{(\mathbf{w}, \underline{\varepsilon})} (h_{\mathbf{w}}(x + \underline{\varepsilon}) \in \mathcal{R}_m) \geq \check{p}_{\mathcal{R}_m}$ for a convex set $\mathcal{R}_m \subset \mathcal{V}_m$. For $r \geq 0$, define*

$$\underline{p}_{\mathcal{V}_m} = \Phi \left(\Phi^{-1}(\check{p}_{\mathcal{V}_m}) - \frac{r}{\sigma} \right), \quad \bar{p}_{\mathcal{V}_m} = \Phi \left(\Phi^{-1}(\hat{p}_{\mathcal{V}_m}) + \frac{r}{\sigma} \right), \quad \text{and} \quad \underline{p}_{\mathcal{R}_m} = \Phi \left(\Phi^{-1}(\check{p}_{\mathcal{R}_m}) - \frac{r}{\sigma} \right). \quad (11)$$

Then, for any $\delta \in \mathbb{R}^d$ such that $\|\delta\|_2 \leq r$, it holds that

$$\begin{aligned} & \mathbb{P}_{(\mathbf{w}, \underline{\varepsilon})} \left(\tilde{\mathcal{H}}_{N, \alpha, \mathbf{V}}(x + \delta) \in \mathcal{R}_m \right) \\ & \geq \sum_{s=1}^N \frac{s}{N} \sum_{j=s-\lfloor \alpha s \rfloor}^s f_{s,j} \left(\frac{\underline{p}_{\mathcal{R}_m}}{\bar{p}_{\mathcal{V}_m}} \right) \min \left\{ f_{N,s} \left(\underline{p}_{\mathcal{V}_m} \right), f_{N,s} \left(\bar{p}_{\mathcal{V}_m} \right) \right\}. \end{aligned} \quad (12)$$

Proof. First, we observe that using the Neyman-Pearson's Lemma [8], for any $\delta \in \mathbb{R}^d$ such that $\|\delta\|_2 \leq r$,

$$\mathbb{P}_{(\mathbf{w}, \underline{\varepsilon})} (h_{\mathbf{w}}(x + \delta + \underline{\varepsilon}) \in \mathcal{V}_m) \geq \Phi \left(\Phi^{-1}(\check{p}_{\mathcal{V}_m}) - \frac{\|\delta\|_2}{\sigma} \right) \geq \Phi \left(\Phi^{-1}(\check{p}_{\mathcal{V}_m}) + \frac{r}{\sigma} \right) = \underline{p}_{\mathcal{V}_m}, \quad (13)$$

$$\mathbb{P}_{(\mathbf{w}, \underline{\varepsilon})} (h_{\mathbf{w}}(x + \delta + \underline{\varepsilon}) \in \mathcal{V}_m) \leq \Phi \left(\Phi^{-1}(\hat{p}_{\mathcal{V}_m}) + \frac{\|\delta\|_2}{\sigma} \right) \leq \Phi \left(\Phi^{-1}(\hat{p}_{\mathcal{V}_m}) - \frac{r}{\sigma} \right) = \bar{p}_{\mathcal{V}_m}, \quad (14)$$

and

$$\mathbb{P}_{(\mathbf{w}, \underline{\varepsilon})} (h_{\mathbf{w}}(x + \delta + \underline{\varepsilon}) \in \mathcal{R}_m) \geq \Phi \left(\Phi^{-1}(\check{p}_{\mathcal{R}_m}) - \frac{\|\delta\|_2}{\sigma} \right) \geq \Phi \left(\Phi^{-1}(\check{p}_{\mathcal{R}_m}) - \frac{r}{\sigma} \right) = \underline{p}_{\mathcal{R}_m}, \quad (15)$$

due to the monotonicity of the function Φ . Further, because by construction $\mathcal{R}_m \subset \mathcal{V}_m$, we have that

$$\mathbb{P}_{(\mathbf{w}, \underline{\varepsilon})} (h_{\mathbf{w}}(x + \delta + \underline{\varepsilon}) \in \mathcal{R}_m \mid h_{\mathbf{w}}(x + \delta + \underline{\varepsilon}) \in \mathcal{V}_m) \geq \frac{\underline{p}_{\mathcal{R}_m}}{\bar{p}_{\mathcal{V}_m}}. \quad (16)$$

Having these probability bounds for $h_{\mathbf{w}}(x + \delta + \underline{\varepsilon})$, we can now proceed to study the probability of the event $\tilde{\mathcal{H}}_{N, \alpha, \mathbf{V}}(x + \delta) \in \mathcal{R}_m$. For any $x' \in \mathcal{X}$, by applying the Law of Total Probability twice, it holds that

$$\begin{aligned} & \mathbb{P}_{(\mathbf{w}, \underline{\varepsilon})} \left(\tilde{\mathcal{H}}_{N, \alpha, \mathbf{V}}(x') \in \mathcal{R}_m \right) = \\ & \mathbb{P}_{(\mathbf{w}, \underline{\varepsilon})} \left(\tilde{\mathcal{H}}_{N, \alpha, \mathbf{V}}(x') \in \mathcal{R}_m \mid \tilde{\mathcal{H}}_{N, \alpha, \mathcal{V}_m}(x') \in \mathcal{R}_m \right) \mathbb{P}_{(\mathbf{w}, \underline{\varepsilon})} \left(\tilde{\mathcal{H}}_{N, \alpha, \mathcal{V}_m}(x') \in \mathcal{R}_m \right) + \\ & \underbrace{\mathbb{P}_{(\mathbf{w}, \underline{\varepsilon})} \left(\tilde{\mathcal{H}}_{N, \alpha, \mathbf{V}}(x') \in \mathcal{R}_m \mid \tilde{\mathcal{H}}_{N, \alpha, \mathcal{V}_m}(x') \notin \mathcal{R}_m \right) \mathbb{P}_{(\mathbf{w}, \underline{\varepsilon})} \left(\tilde{\mathcal{H}}_{N, \alpha, \mathcal{V}_m}(x') \notin \mathcal{R}_m \right)}_{=0} = \\ & \sum_{s=0}^N \mathbb{P}_{(\mathbf{w}, \underline{\varepsilon})} \left(\tilde{\mathcal{H}}_{N, \alpha, \mathbf{V}}(x') \in \mathcal{R}_m \mid \tilde{\mathcal{H}}_{N, \alpha, \mathcal{V}_m}(x') \in \mathcal{R}_m, |I_m| = s \right) \cdot \\ & \mathbb{P}_{(\mathbf{w}, \underline{\varepsilon})} \left(\tilde{\mathcal{H}}_{N, \alpha, \mathcal{V}_m}(x') \in \mathcal{R}_m \mid |I_m| = s \right) \mathbb{P}_{(\mathbf{w}, \underline{\varepsilon})} (|I_m| = s). \end{aligned} \quad (17)$$

In particular, let $x' = x + \delta$. We will examine each of these terms separately. Recall that, by construction, the random set of indexes I_m is given by $I_m = \{i \in [N] : h_{\mathbf{w}}(x + \delta + \varepsilon_i) \in \mathcal{V}_m\}$, so that $|I_m| \sim \text{Binomial}(N, \mathbb{P}_{(\mathbf{w}, \underline{\varepsilon})} (h_{\mathbf{w}}(x + \delta + \underline{\varepsilon}) \in \mathcal{V}_m))$. Therefore,

$$\mathbb{P}_{(\mathbf{w}, \underline{\varepsilon})} (|I_m| = s) \geq \min \left\{ f_{N,s} \left(\underline{p}_{\mathcal{V}_m} \right), f_{N,s} \left(\bar{p}_{\mathcal{V}_m} \right) \right\}, \quad (18)$$

as $\mathbb{P}_{(\mathbf{w}, \varepsilon)}(h_{\mathbf{w}}(x + \delta + \varepsilon) \in \mathcal{V}_m) \in [\underline{p}_{\mathcal{V}_m}, \bar{p}_{\mathcal{V}_m}]$ by (13) and (14). That the minimum is attained at $\underline{p}_{\mathcal{V}_m}$ or $\bar{p}_{\mathcal{V}_m}$ follows from the fact that the continuous function $f_{N,s}(p) = \binom{N}{s} p^s (1-p)^{N-s}$ has a positive derivative $f'(p) > 0$ for $p < \frac{s}{N}$ (resp. $f'(p) < 0$ for $p > \frac{s}{N}$). Hence, $p = \frac{s}{N}$ is the maximizer within $[0, 1]$ and the minimum of $f(p)$ for $p \in [\underline{p}, \bar{p}]$ must be attained at either $p = \underline{p}$ or $p = \bar{p}$.

We then move on to the event $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}}(x + \delta) \in \mathcal{R}_m$ given $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}(x + \delta) \in \mathcal{R}_m, |I_m| = s$. Trivially, conditioned on $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}(x + \delta) \in \mathcal{R}_m$, the event $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}}(x + \delta) \in \mathcal{R}_m$ takes place if the component $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}$ is sampled from the mixture, which by construction is equal to $\frac{|I_m|}{N}$. Thus,

$$\mathbb{P}_{(\mathbf{w}, \varepsilon)} \left(\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}}(x + \delta) \in \mathcal{R}_m \mid \tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}(x + \delta) \in \mathcal{R}_m, |I_m| = s \right) = \frac{s}{N}. \quad (19)$$

Finally, to conclude, we need to lower-bound the probability of the event $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}(x + \delta) \in \mathcal{R}_m$ given $|I_m| = s$. We observe that conditioned on the knowledge $|I_m| = s$, the random variable $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}(x + \delta)$ conditioned on $|I_m| = s$ behaves as the α -smoother introduced in [30] with number of samples s . Therefore, we follow a similar proof. Let $Z_m = \sum_{i=1}^s \mathbf{1}_{h_{\mathbf{w}}(x + \delta + \varepsilon_i) \in \mathcal{R}_m \mid h_{\mathbf{w}}(x + \delta + \varepsilon_i) \in \mathcal{V}_m}$, thus

$$Z_m \sim \text{Binomial}(s, \mathbb{P}_{(\mathbf{w}, \varepsilon)}(h_{\mathbf{w}}(x + \delta + \varepsilon) \in \mathcal{R}_m \mid h_{\mathbf{w}}(x + \delta + \varepsilon) \in \mathcal{V}_m)). \quad (20)$$

From the Law of Total Probability, it holds that

$$\begin{aligned} \mathbb{P}_{(\mathbf{w}, \varepsilon)} \left(\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}(x + \delta) \in \mathcal{R}_m \mid |I_m| = s \right) = \\ \sum_{z=0}^s \mathbb{P}_{(\mathbf{w}, \varepsilon)} \left(\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}(x + \delta) \in \mathcal{R}_m \mid |I_m| = s, Z_m = z \right) \mathbb{P}(Z_m = z), \end{aligned} \quad (21)$$

Now, note that the event $z \geq s - \lfloor \alpha s \rfloor$ means that the α -trimming only discards points out of $\mathcal{R}_m$ (i.e., the average in (2) only contains points in $\mathcal{R}_m$), thus it must hold that $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}(x + \delta) \in \mathcal{R}_m$ conditioned on the given events by the convexity of the set $\mathcal{R}_m$. On the other hand, when $z < s - \lfloor \alpha s \rfloor$, the average in (2) contains points out of $\mathcal{R}_m$. In the absence of more information about how far those points are, we have no guarantees that the average over them will belong to $\mathcal{R}_m$ (as only one point suffices to displace the average arbitrarily far from $\mathcal{R}_m$ [31]). Therefore, conditioned on these given events, we may conservatively consider that $\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}(x + \delta) \notin \mathcal{R}_m$. Thus, from (21),

$$\begin{aligned} \mathbb{P}_{(\mathbf{w}, \varepsilon)} \left(\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}(x + \delta) \in \mathcal{R}_m \mid |I_m| = s \right) = \\ \sum_{z=0}^{s - \lfloor \alpha s \rfloor - 1} \underbrace{\mathbb{P}_{(\mathbf{w}, \varepsilon)} \left(\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}(x + \delta) \in \mathcal{R}_m \mid |I_m| = s, Z_m = z \right)}_{\geq 0} \mathbb{P}_{Z_m}(Z_m = z) + \\ \sum_{z=s - \lfloor \alpha s \rfloor}^s \underbrace{\mathbb{P}_{(\mathbf{w}, \varepsilon)} \left(\tilde{\mathcal{H}}_{N,\alpha,\mathcal{V}_m}(x + \delta) \in \mathcal{R}_m \mid |I_m| = s, Z_m = z \right)}_{=1} \mathbb{P}_{Z_m}(Z_m = z) \\ \geq \sum_{z=s - \lfloor \alpha s \rfloor}^s \mathbb{P}_{Z_m}(Z_m = z) \\ \geq \sum_{j=s - \lfloor \alpha s \rfloor}^s f_{s,j} \left(\frac{p_{\mathcal{R}_m}}{\bar{p}_{\mathcal{V}_m}} \right), \end{aligned} \quad (22)$$

from the fact that Z_m is distributed as the binomial in (20) and its success probability is lower bounded by (16). The proof is concluded by replacing (18), (19), and (22) in (17). $\square$

We are now ready to prove Theorem 3.1.

Proof. By assumption, we have $\mathcal{R}_l \cap \mathcal{R}_{l'} = \emptyset$ for $l, l' \in \mathcal{L}$ where $l \neq l'$. Therefore, we can employ a union-bound argument over Lemma A.1 for the following bound

$$\begin{aligned}
& \mathbb{P}_{(\mathbf{w}, \underline{\epsilon})} \left(\tilde{\mathcal{H}}_{N, \alpha, \mathbf{V}}(x + \delta) \in \tilde{\mathcal{R}} \right) \\
&= \sum_{l \in \mathcal{L}} \mathbb{P}_{(\mathbf{w}, \underline{\epsilon})} \left(\tilde{\mathcal{H}}_{N, \alpha, \mathbf{V}}(x + \delta) \in \mathcal{R}_l \right) \\
&\geq \sum_{l \in \mathcal{L}} \inf_{p_{V_l} \in [\underline{p}_{V_l}, \bar{p}_{V_l}]} \sum_{s=1}^N \frac{s}{N} \sum_{j=s-\lfloor \alpha s \rfloor}^s f_{s,j} \left(\frac{p_{\mathcal{R}_l}}{p_{V_l}} \right) f_{N,s}(p_{V_l}) \\
&= \inf_{p_{V_m} \in [\underline{p}_{V_m}, \bar{p}_{V_m}]} \sum_{l \in \mathcal{L}} \sum_{s=1}^N \frac{s}{N} \sum_{j=s-\lfloor \alpha s \rfloor}^s f_{s,j} \left(\frac{p_{\mathcal{R}_l}}{p_{V_l}} \right) f_{N,s}(p_{V_l}), \tag{23}
\end{aligned}$$

where the last infimum is for all $m \in [M]$. Naturally, one would compute this infimum independently per $l \in \mathcal{L}$. However, extracting it reveals that we can add the constraint $\sum_{m=1}^M p_{V_m} = 1$, i.e., ensure that the probability over all partition cells $\mathcal{V}_m$ sum to one. Thus, we arrive at

$$\begin{aligned}
& \mathbb{P}_{(\mathbf{w}, \underline{\epsilon})} \left(\tilde{\mathcal{H}}_{N, \alpha, \mathbf{V}}(x + \delta) \in \tilde{\mathcal{R}} \right) \\
&\geq \inf_{\substack{p_{V_m} \in [\underline{p}_{V_m}, \bar{p}_{V_m}] \\ \sum_{m=1}^M p_{V_m} = 1}} \sum_{l \in \mathcal{L}} \sum_{s=1}^N \frac{s}{N} \sum_{j=s-\lfloor \alpha s \rfloor}^s f_{s,j} \left(\frac{p_{\mathcal{R}_l}}{p_{V_l}} \right) f_{N,s}(p_{V_l}), \tag{24}
\end{aligned}$$

which concludes the proof. $\square$

A.2 Proof of Proposition 3.2

Before proving Proposition 3.2, we first prove an auxiliary Lemma.

Lemma A.2. *Let $g : [\ell, u] \rightarrow \mathbb{R}$ be a Lipschitz continuous function with Lipschitz constant $\mathcal{L}_g$. Let $\{p^{(1)}, \dots, p^{(K)}\}$ be a set of $K \in \mathbb{N}_{>0}$ anchor points such that $\ell = p^{(1)} < p^{(2)} < \dots < p^{(K)} = u$. Let $\text{conv}_K(g)(p)$ be defined as*

$$\text{conv}_K(g)(p) = \min_{\lambda \in \Delta_K} \left\{ \sum_{k=1}^K \lambda^{(k)} g(p^{(k)}) \text{ s.t. } \sum_{k=1}^K \lambda^{(k)} p^{(k)} = p \right\}. \tag{25}$$

Then,

$$\text{conv}_K(g)(p) \leq g(p) + \mathcal{L}_g h_K, \tag{26}$$

where $h_K = \max_{p' \in [\ell, u]} \min_{k \in [K]} |p' - p^{(k)}|$.

Proof. For any $p \in [\ell, u]$, there exist consecutive anchor points $p^{(k)}, p^{(k+1)}$ such that $p^{(k)} \leq p \leq p^{(k+1)}$. We can write p as a convex combination of those anchor points, i.e. $p = \lambda p^{(k)} + (1 - \lambda) p^{(k+1)}$ by taking $\lambda = \frac{p^{(k+1)} - p}{p^{(k+1)} - p^{(k)}} \in [0, 1]$. Thus, the vector $\lambda = [0, \dots, 0, \lambda, 1 - \lambda, 0, \dots, 0]$ (i.e., non-zero at positions k and $k+1$) belongs to Δ_K and satisfies the constraint in (25), thus

$$\text{conv}_K(g)(p) \leq \lambda g(p^{(k)}) + (1 - \lambda) g(p^{(k+1)}). \tag{27}$$

Let $h_K = \max_{p \in [\ell, u]} \min_{k \in [K]} |p - p^{(k)}|$. Then,

$$\begin{aligned}
\lambda g(p^{(k)}) + (1 - \lambda) g(p^{(k+1)}) &\leq \lambda g(p) + (1 - \lambda) g(p) + \lambda |g(p^{(k)}) - g(p)| + (1 - \lambda) |g(p^{(k+1)}) - g(p)| \\
&\leq g(p) + L_g h_K. \tag{28}
\end{aligned}$$

where L_g is the Lipschitz constant of g in $[\ell, u]$. Thus, from (27) and (28), we have that, for any $p \in [\ell, u]$,

$$\text{conv}_K(g)(p) \leq g(p) + L_g h_K.$$

$\square$

We are now ready to prove Proposition 3.2.

Proof. The goal of Proposition 3.2 is to bound the following optimization problem.

$$\inf_{\substack{p_{\mathcal{V}_m} \in [\underline{p}_{\mathcal{V}_m}, \bar{p}_{\mathcal{V}_m}] \\ \sum_{m=1}^M p_{\mathcal{V}_m} = 1}} \sum_{l \in \mathcal{L}} \sum_{s=1}^N \frac{s}{N} \sum_{j=s-\lfloor \alpha s \rfloor}^s f_{s,j} \left(\frac{\underline{p}_{\mathcal{R}_l}}{p_{\mathcal{V}_l}} \right) f_{N,s}(p_{\mathcal{V}_l}). \quad (29)$$

Due to the non-convexity of (29), we will search for a tractable convex lower-bound. First, note that the objective function is separable in $p_{\mathcal{V}_l}$ as a sum of functions g_l , which allows the objective in (29) to be written as $G(\mathbf{p}) = \sum_{l \in \mathcal{L}} g_l(p_{\mathcal{V}_l})$, where $\mathbf{p} = \{p_{\mathcal{V}_l}\}_{l \in \mathcal{L}}$. For each component l , select K anchor points:

$$p_{\mathcal{V}_l}^{(1)} < p_{\mathcal{V}_l}^{(2)} < \dots < p_{\mathcal{V}_l}^{(K)}, \quad p_{\mathcal{V}_l}^{(k)} \in [\underline{p}_{\mathcal{V}_l}, \bar{p}_{\mathcal{V}_l}], \quad (30)$$

and evaluate $g_l^{(k)} := g_l(p_{\mathcal{V}_l}^{(k)})$ at each anchor. Then, let

$$\text{conv}_K(g_l)(p) = \min_{\boldsymbol{\lambda} \in \Delta_K} \left\{ \sum_{k=1}^K \lambda^{(k)} g_l(p^{(k)}) \text{ s.t. } \sum_{k=1}^K \lambda^{(k)} p^{(k)} = p \right\}. \quad (31)$$

From Lemma A.2, it holds that

$$\text{conv}_K(g_l)(p_{\mathcal{V}_l}) \leq g_l(p_{\mathcal{V}_l}) + \mathcal{L}_{g_l} h_{l,K}, \quad (32)$$

where $\mathcal{L}_{g_l}$ is the Lipschitz constant of the function g_l in the interval $[\underline{p}_{\mathcal{V}_l}, \bar{p}_{\mathcal{V}_l}]$ and the maximum interval size $h_{l,K} = \max_{p \in [\underline{p}_{\mathcal{V}_l}, \bar{p}_{\mathcal{V}_l}]} \min_{k \in [K]} |p - p_{\mathcal{V}_l}^{(k)}|$. Therefore,

$$\inf_{\substack{p_{\mathcal{V}_m} \in [\underline{p}_{\mathcal{V}_m}, \bar{p}_{\mathcal{V}_m}] \\ \sum_{m=1}^M p_{\mathcal{V}_m} = 1}} \sum_{l \in \mathcal{L}} \text{conv}_K(g_l)(p_{\mathcal{V}_l}) \leq \inf_{\substack{p_{\mathcal{V}_m} \in [\underline{p}_{\mathcal{V}_m}, \bar{p}_{\mathcal{V}_m}] \\ \sum_{m=1}^M p_{\mathcal{V}_m} = 1}} \sum_{l \in \mathcal{L}} g_l(p_{\mathcal{V}_l}) + \sum_{l \in \mathcal{L}} \mathcal{L}_{g_l} h_{l,K}. \quad (33)$$

As the $\boldsymbol{\lambda}$ variables within the definition of each $\text{conv}_K(g_l)(p_{\mathcal{V}_l})$ are independent from each other, we can write the left-hand side term as the following LP problem

$$\begin{aligned} \inf_{\substack{p_{\mathcal{V}_m} \in [\underline{p}_{\mathcal{V}_m}, \bar{p}_{\mathcal{V}_m}] \\ \sum_{m=1}^M p_{\mathcal{V}_m} = 1}} \sum_{l \in \mathcal{L}} \text{conv}_K(g_l)(p_{\mathcal{V}_l}) &= \inf_{\mathbf{p}, \boldsymbol{\lambda}_1, \dots, \boldsymbol{\lambda}_{|\mathcal{L}|}} \sum_{l \in \mathcal{L}} \sum_{k=1}^K \lambda_l^{(k)} g_l(p_{\mathcal{V}_l}^{(k)}) \\ \text{s.t. } &\begin{cases} p_{\mathcal{V}_m} \in [\underline{p}_{\mathcal{V}_m}, \bar{p}_{\mathcal{V}_m}], \\ \sum_{m=1}^M p_{\mathcal{V}_m} = 1, \\ \sum_{k=1}^K \lambda_l^{(k)} p_{\mathcal{V}_l}^{(k)} = p_{\mathcal{V}_l}, \\ \boldsymbol{\lambda}_l \in \Delta_K. \end{cases} \end{aligned} \quad (34)$$

To conclude, we apply Lemma A.3 to obtain

$$L_{g_l} \leq \frac{N}{1 - \bar{p}_{\mathcal{V}_l}} + \frac{N}{\underline{p}_{\mathcal{V}_l} - \underline{p}_{\mathcal{R}_l}}.$$

□

Lemma A.3. Let $g : (0, 1] \rightarrow \mathbb{R}$ be the function defined by

$$g(p) = \sum_{s=1}^N \frac{s}{N} \sum_{j=s-\lfloor \alpha s \rfloor}^s \binom{s}{j} \left(\frac{\tilde{p}}{p} \right)^j \left(1 - \frac{\tilde{p}}{p} \right)^{s-j} \binom{N}{s} p^s (1-p)^{N-s},$$

where $\alpha \in [0, \frac{1}{2}]$, $N \in \mathbb{N}_{>0}$, and $0 < \tilde{p} < \underline{p} < \bar{p} < 1$. Then g is Lipschitz continuous on $[\underline{p}, \bar{p}]$ with Lipschitz constant

$$L_g \leq \frac{N}{1 - \bar{p}} + \frac{N}{\underline{p} - \tilde{p}}. \quad (36)$$

Proof. Because g is a sum of continuous and differentiable functions on $[\underline{p}, \bar{p}]$, its Lipschitz constant is given by $L_g = \sup_{p \in [\underline{p}, \bar{p}]} |g'(p)|$. For notational simplicity, we write the summand as

$$T_{s,j}(p) = C_{s,j} \left(\frac{\tilde{p}}{p}\right)^j \left(1 - \frac{\tilde{p}}{p}\right)^{s-j} p^s (1-p)^{N-s}, \quad C_{s,j} = \binom{s}{j} \binom{N}{s}.$$

First, note that, for $p^s (1-p)^{N-s}$, $\left| \frac{d}{dp} [p^s (1-p)^{N-s}] \right| \leq p^s (1-p)^{N-s} \frac{N}{1-\tilde{p}}$. Then, for $(\tilde{p}/p)^j (1 - \tilde{p}/p)^{s-j}$, the absolute value of the derivative is at most $(\tilde{p}/p)^j (1 - \tilde{p}/p)^{s-j} \frac{N}{\underline{p} - \tilde{p}}$. Since all coefficients are nonnegative and sum to at most 1, it holds that

$$|g'(p)| \leq \frac{N}{1-\tilde{p}} + \frac{N}{\underline{p} - \tilde{p}}, \quad p \in [\underline{p}, \bar{p}],$$

which concludes the proof. $\square$

A.3 Proof of Proposition 4.1

Before proving Proposition 4.1, we introduce the well-known Clopper-Pearson Lemma.

Lemma A.4 (Clopper-Pearson lower bound). *Let $\{\mathbf{z}_1, \dots, \mathbf{z}_M\}$ be i.i.d. samples from $\mathbb{P}$, and let $h : \mathcal{X} \rightarrow \mathcal{Y}$ and $\mathcal{R} \subseteq \mathcal{Y}$ be given. Define $Z = \sum_{i=1}^M \mathbf{1}_{h(\mathbf{z}_i) \in \mathcal{R}}$. Given a confidence level $\beta \in [0, 1]$, if $\tilde{p} \in [0, 1]$ is such that $\beta = \sum_{j=Z}^M f_{M,j}(\tilde{p})$, then*

$$\mathbb{P}_{(\mathbf{z}_1, \dots, \mathbf{z}_M)} (\mathbb{P}_{\mathbf{z}} (h(\mathbf{z}) \in \mathcal{R}) \geq \tilde{p}) \geq 1 - \beta \quad (37)$$

We are now ready to prove Proposition 4.1.

Proof. From the Fréchet bound and Lemma A.4,

$$\begin{aligned} & \mathbb{P}_{\{(\mathbf{w}_i, \varepsilon_i)\}_{i=1}^N} \left(\bigcap_{m=1}^M \{ \check{p}_{\mathcal{V}_m} \leq \mathbb{P}_{(\mathbf{w}, \varepsilon)} (h_{\mathbf{w}}(x + \varepsilon) \in \mathcal{V}_m) \leq \hat{p}_{\mathcal{V}_m}, \check{p}_{\mathcal{R}_m} \leq \mathbb{P}_{(\mathbf{w}, \varepsilon)} (h_{\mathbf{w}}(x + \varepsilon) \in \mathcal{R}_m) \} \right) \\ & \geq \sum_{i=1}^{3M} \left(1 - \frac{\beta}{3M} \right) - (3M - 1) = 1 - \beta. \end{aligned}$$

$\square$

B Additional Experimental Results

B.1 Trajectory Prediction

We examine a specific input instance from the dataset and demonstrate that the introduction of a small input perturbation $\varepsilon \sim \mathcal{N}(0, I)$ increases the behavioral variability of the target agent by raising the probability of the tactical behavior *go*. While α -smoothing tends to suppress the *go* mode – thereby suggesting that the target vehicle is likely to yield – our clustered α -trimming preserves the predicted multi-modality of $h_{\mathbf{w}}(x + \varepsilon)$, as illustrated in Figure 7, where we show the last time step of the sampled trajectories.

C Summary of method parameters

The clustered α -smoothing certificate is parameterized by three groups. The clustering group depends on the strategy. For the purposes of the experiments in Section 5, we use DBSCAN. The parameters are listed in Table 2.

D Experiment Details

D.1 Trajectory Prediction

TrajFlow architecture and training procedure. *TrajFlow* [26] is built with a combination of a normalizing flow network F with a recurrent autoencoder, consisting of encoder E_{RNN} and decoder

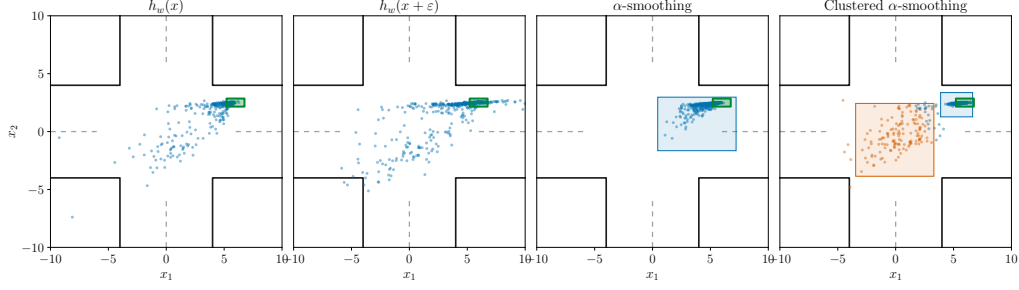


Figure 7: Given a particular trajectory input x from the L-GAP dataset [47], we show the last time step of trajectory samples from $h_w(x)$, $h_w(x + \varepsilon)$, where $\varepsilon \sim \mathcal{N}(0, I)$, the α -smoothing in (1), and our clustered α -smoothing. We also show the projection to the last time step subspace of the sets $\mathcal{R}_m$ produced with Algorithm 1 for coverage $p = 0.8$.

Table 2: Clustered α -smoothing parameters.

Parameter	Symbol	Impact
<i>Clustering (DBSCAN strategy)</i>		
Maximum number of clusters	$M_{\max}$	If the output of DBSCAN has more than $M_{\max}$ clusters, then we merge them in order of fewest to most samples.
Coverage	p	See Line 9 of Algorithm 1
Neighborhood radius	ε_{db}	DBSCAN neighbourhood radius in output space.
Min. samples	—	DBSCAN core-point threshold.
<i>Smoothing core</i>		
Std.dev. of Gaussian input-perturbation	σ	Larger std.dev. allows larger perturbation radius r , at the cost of larger deviation from $h_w(x)$. See (4).
Number of samples for smoothing	N	More samples tightens the modes of the smoothed distribution.
Trimming fraction	α	The certificate discards the most extreme 2α tails of each per-cluster output coordinate, $0 \leq \alpha < 1/2$.
<i>Confidence</i>		
Number of samples for confidence	$\bar{N}$	Number of samples used for Proposition 4.1. More samples leads to better confidence, at an increased computational cost.
Confidence parameter	β	The joint confidence for all estimated probability bounds is $1 - \beta$.

E_{RNN} (see Figure 8). Furthermore, the model is completed with the networks ϕ_{RNN} (a gated recurring unit encoding the predicted agents past trajectory), ψ_{GNN} (a graph neural network encoding the other agents), and ϕ_{CNN} (a convolutional network encoding the environment, which is given as a bird’s-eye image). This model is trained in a two-stage process, where first the recurrent autoencoder is trained to encode the future trajectories of all involved agents. Once done, the normalizing flow is trained on the encoded future sampled as well as the past and environment information. An detailed description of the model components and the training loss can be found in the original work [26].

For model training, we employ the *STEP* framework [38] (MIT License) to train *TrajFlow* for our chosen scenario. In particular, the model is trained on a dataset combining *NuScenes* [5] (CC BY-NC-SA) and *L-GAP* [47] (CC-BY). Following previous works [37, 36], we do not use the standard input for *NuScenes* (i.e., 4 past time steps at a frequency of 2 Hz), but instead use *STEP* to extract past trajectories with 10 time steps at 10 Hz. This choice is mostly motivated by the fact that this results in a smoother trajectory, for which we can extract the underlying control states much easier. Importantly, while these datasets provide trajectories with states including positions, velocities and accelerations, for *TrajFlow*, we only consider the recorded positions. The code is available at https://github.com/DAI-Lab-HEReALD/General-Framework/blob/main/Framework/simulations_randomized_smoothing.py.

Behavioral clustering. For our experiments, we run Algorithm 1 with a behavior-based two-cluster heuristic in endpoint space, using prior knowledge of two tactical modes (*stay* and *go*). For each sample, let $\Delta = y_{\text{end}} - y_{\text{init}}$. A sample is tagged as *stay* when $\|\Delta\|_2 \leq r_{\text{stay}}$, and as *go* when $\Delta^{(2)} \leq -\delta_{\downarrow}$ and $\Delta^{(1)} \leq -\delta_{\leftarrow}$, excluding already-tagged *stay* samples. Endpoint prototypes are then

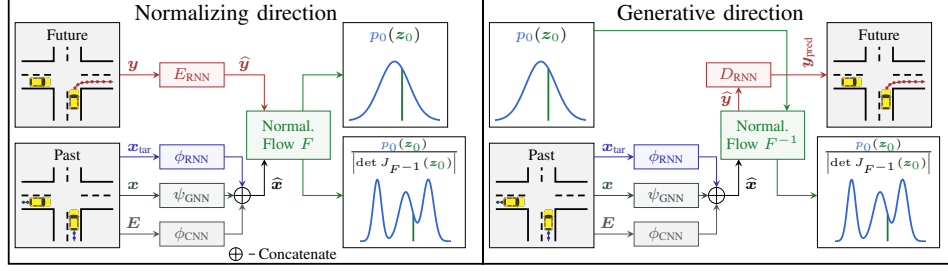


Figure 8: An overview of *TrajFlow*'s architecture, taken from Figure 2 in [26]. During training, future trajectories y are encoded with the encoder E_{RNN} and transformed to the abstracted features $\hat{y}$. Using the normalizing direction, they are then transformed into a sample $z_0 = F(\hat{y})$, which follows a standard normal distribution p_0 . For inference we then use the generative direction, in which a sample $z_0 \sim p_0$ is inversely transformed by the Normalizing Flow to generate the abstracted future trajectories $\hat{y} = F^{-1}(z_0)$ that are decoded with D_{RNN} into the actual trajectories y_{pred} . The likelihood of the encoded trajectory $\hat{y}$ is obtained with $p_0(z_0)|\det J_{F^{-1}}(z_0)|^{-1}$. The encoding ϕ_{CNN} of map E , and the encoding ψ_{GNN} of social interactions are optional blocks, which can provide richer context information.

built from these tagged sets (with fallback rules if one set is empty), and all samples are assigned to the nearest prototype in 2D endpoint space. Finally, for each cluster $j \in 1, 2$, we construct a tight axis-aligned hyperrectangle $\mathcal{R}_j$ in the full trajectory space (40D) that covers a fraction p of the samples assigned to that cluster.

D.2 Quadrotor Control

The quadrotor control benchmark was first described in [3]. We use it solely to obtain a safety-critical, stochastic, multi-modal closed-loop system on which to perform clustered randomized smoothing. Relative to [3], we augment the system with a reward structure and extended termination criteria.

Environment. The state is $s_k = [x_k, \dot{x}_k, y_k, \dot{y}_k, z_k, \dot{z}_k] \in \mathbb{R}^6$ and the action $u_k = [u_k^x, u_k^y, u_k^z] \in [-4, 4]^3$. The dynamics are a per-axis double integrator with unit time step,

$$s_{k+1} = As_k + Bu_k + Cw_k, \quad w_k \sim \mathcal{N}(0, \sigma^2 I) \quad (38)$$

with velocities clipped to $[-7, 7]$ after each update. In particular, per axis the dynamics are $x_{k+1} = x_k + \tau \dot{x}_k + \frac{1}{2} \tau^2 u_k^x$ and $\dot{x}_{k+1} = \dot{x}_k + \tau u_k^x + w_k$ where $w_k \sim \mathcal{N}(0, \sigma^2)$. The corridor is $[-15, 15] \times [-9, 9] \times [-7, 7]$, the goal region is $[11, 15] \times [1, 5] \times [-7, -3]$, and 14 axis-aligned hyperrectangular obstacles match [3]. The termination criteria are *in order*:

1. if $[x_k, y_k, z_k]$ is outside the corridor,
2. if *any* point on the line segment between $[x_k, y_k, z_k]$ and $[x_{k+1}, y_{k+1}, z_{k+1}]$ touches an obstacle,
3. if $[x_k, y_k, z_k]$ is inside the goal region, or
4. if the max episode length is reached.

The parameters of the environment are summarized in Table 3.

Policy and value networks. The policy is a tanh-squashed Gaussian mixture: at state s , sample a mode $z \sim \text{Cat}(\text{softmax}(g_\phi(s)))$ where Cat is a categorical distribution, sample an unscaled action from this mode $r \sim \mathcal{N}(\mu_\theta^z(s), \text{diag } e^{2 \log \sigma_z})$, and rescale the action $a = \tanh(r) \cdot a_{\text{scale}}$. The mixture results in a multi-modal policy, which indeed requires *clustered* α -smoothing. The architecture is summarized in Table 4.

Training. The policy is trained using standard clipped PPO with GAE and a curriculum on the start state. The curriculum using a linear schedule where a scalar $f \in [0, 1]$ linearly anneals from 0 to 1 over the first 40,000 episodes. At each rollout we sample a valid (collision-free, outside the goal) start position uniformly inside an axis-aligned box around the goal center with half-width $r = 3 + 25f$, and initial velocities $\mathcal{N}(0, (0.1 + 0.4f)^2)$. Without curriculum learning, the policy

Table 3: Environment parameters. Δdist denotes the change in distance $\|\text{pos} - \text{goal}\|$ to the goal center $\text{goal} = (13, 3, -5)$ between subsequent steps.

Quantity	Value	Notes
State / action dim	6 / 3	
Action bounds	$[-4, 4]^3$	matches JAIR benchmark
Process noise σ	0.05	on velocity coords; small enough that obstacles dominate
Velocity clip	$[-7, 7]$	prevents PPO blow-up under exploration
Max steps / episode	64	matches rollout horizon
Reward (goal)	+10	terminal
Reward (collision / exit)	-5	terminal
Reward (timeout)	$-0.05 \cdot \ \text{pos} - \text{goal}\ $	distance-shaped: far timeouts penalised more
Reward (alive)	$0.5 \cdot \Delta\text{dist} - 0.01$	dense progress shaping
State normalization	$s/[15, 7, 9, 7, 7, 7]$	per-dim rescale for the policy net

Table 4: Network details. A denotes the action dimension.

Component	Specification
Trunk	$2 \times \text{Linear}(128) + \tanh$, orthogonal init (gain $\sqrt{2}$)
Mean head	$\text{Linear}(K \cdot A)$, orthogonal init gain 0.01
Mixture head ($K > 1$)	$\text{Linear}(K)$, orthogonal init gain 0.01
Log-std	global per-component $[K, A]$, init -1.2 , clamped to $[-5, 2]$
Critic	identical $2 \times \text{Linear}(128) + \tanh$ trunk, scalar head
K (mixture components)	2
a_{scale}	4.0 (matches action bound; tanh saturation gives the true control limit)

frequently terminates in collisions before it ever sees the goal. The remaining training parameters are listed in Table 5.

Table 5: PPO hyperparameters.

Hyperparameter	Value	Notes
Optimizer	Adam, lr 3×10^{-4}	shared across actor and critic
Total episodes	200,000	
Rollout horizon T	64 steps	matches env timeout
Episodes per update	32	$\approx 2k$ transitions / batch
PPO epochs / batch	10	
Minibatch size	512	
Discount γ	0.99	
GAE λ	0.95	
Clip range	0.2	
Value loss coefficient	0.5	
Entropy coefficient	0.02	mild; the mixture itself adds exploration
Max grad norm	0.5	global clip across actor + critic
Advantage normalization	per-batch (mean / std)	

Randomized smoothed policy parameters. Figure 6 is produced with the clustered α -smoothing parameters in Table 6 below. α -smoothing uses the same parameters, except for no clustering.

E Generalized Voronoi partition

Given a collection of points $\mathcal{C} = \{c_1, \dots, c_M\} \subset \mathcal{X}$, the Voronoi partition $\tilde{\mathcal{V}}$ induced by $\mathcal{C}$ is defined⁵ as $\tilde{\mathcal{V}}_i = \{x \in \mathcal{X} : \|x - c_i\|_2 \leq \|x - c_j\|_2, i \neq j\}$, i.e., the point x belongs to the cell $\tilde{\mathcal{V}}_i$ if it is closer to c_i than to any other point c_j in $\mathcal{C}$. The Generalized Voronoi partition extend this notion

⁵Although we introduce the Voronoi partition for the Euclidean distance, we must stress that it can be defined for any other metric.

Table 6: Clustered α -smoothing parameters for the quadrotor benchmarks.

Parameter	Value	Notes
σ	0.05	
N	30	
α	0.4	
$M_{\max}$	3	
Coverage p	0.9	
Clustering alg.	DBSCAN	
ε_{db}	0.45	Output-space DBSCAN radius.
dbscan_min_samples	50	Core-point threshold relative.
$\bar{N}$	4000	Samples for Proposition 4.1.
β	10^{-2}	

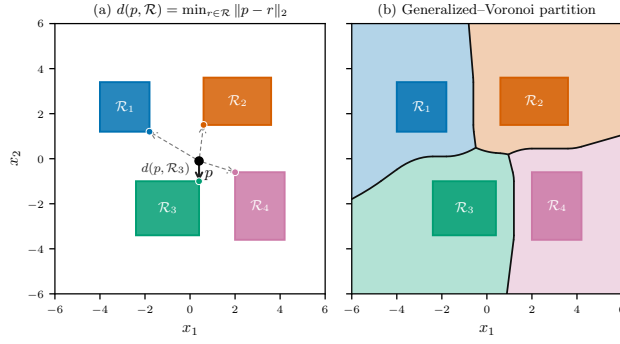


Figure 9: Generalized Voronoi partition of four disjoint axis-aligned rectangles $\mathcal{R}_1, \dots, \mathcal{R}_4 \subset \mathbb{R}^2$ under the Euclidean metric. (a) The distance from point p to the closest point in each region $\mathcal{R}_i$ is measured and p is assigned $\mathcal{V}_3$ as $\mathcal{R}_3$ is the closest. (b) Each cell $\mathcal{V}_i$ collects every point closer to $\mathcal{R}_i$ than to any other site. Cell boundaries in the case of rectangles in 2D consist of straight segments and parabolic arcs and meet at *Voronoi vertices* where three cells agree on the distance to a common point.

to sets, provided a shortest-distance measure from x to a set $\mathcal{R} \subseteq \mathcal{X}$. We write

$$d(x, \mathcal{R}) = \inf_{r \in \mathcal{R}} \|x - r\|_2 \quad (39)$$

for the (Euclidean) point-to-set distance, which equals 0 if and only if $x \in \mathcal{R}$ for closed $\mathcal{R}$.

Definition 4 (Generalized Voronoi partition). *For a collection $\mathcal{R} = \{\mathcal{R}_1, \dots, \mathcal{R}_M\}$ of pairwise disjoint closed sets in $\mathcal{X}$, the generalized Voronoi cell associated with $\mathcal{R}_i$ is given by*

$$\mathcal{V}_i = \{x \in \mathcal{X} : d(x, \mathcal{R}_i) \leq d(x, \mathcal{R}_j) \text{ for all } j \neq i\},$$

and $\mathcal{V} = \{\mathcal{V}_i\}_{i=1}^M$ is the generalized Voronoi partition induced by $\mathcal{R}$.

Unlike $\tilde{\mathcal{V}}$, the cells $\mathcal{V}_i$ in the generalized Voronoi partition $\mathcal{V}$ are generally *not convex*, even when the sets $\mathcal{R}_i$ are convex or simple shapes such as rectangles, as shown in Figure 9. This fact, together with the combinatorial size of the partition that grows rapidly with both the number of sets M and the dimension of the ambient space $\mathcal{X}$, implies that computing the partition is almost certainly intractable. Indeed, even for the point-induced partition $\tilde{\mathcal{V}}$, the worst-case complexity in $\mathcal{X} \subseteq \mathbb{R}^d$ is already $\Theta(M^{\lceil d/2 \rceil})$. When the sets $\mathcal{R}_i$ are convex, the cell boundaries are $(d-1)$ -dimensional algebraic surfaces, and for arbitrary compact sets the topology can be unbounded. Explicit construction of the partition is, therefore, intractable in $\mathbb{R}^d$ beyond toy examples. Crucially, however, our procedure does not require the explicit construction of the partition, but only identifying to which cell $\mathcal{V}_i$ a given point $x \in \mathcal{X}$ belongs, which reduces to computing M point-to-set distances (39). This admits a trivial implementation when, for instance, the sets $\mathcal{R}_i$ are axis-aligned hyperrectangles.

F Quantization artifact in α -smoothing

As shown in Figure 6, α -smoothing on bi-modal distributions unintuitively outputs a “quantized” smoothed prediction. The prediction concentrates on a small, near-uniformly spaced set of *quantization levels* whose positions and weights are fully determined by the mixture and trimming parameters.

We describe this artifact with the simplest non-trivial case, a 1-D two-component Gaussian mixture, which is enough to expose the mechanism. The same reasoning extends mode-by-mode to mixtures with more components or to higher-dimensional outputs (where each axis is quantized independently after coordinate-wise sorting).

F.1 Setup

Let the output distribution be the bi-modal Gaussian mixture

$$p(y) = \pi_A \mathcal{N}(y \mid \mu_A, \sigma^2) + \pi_B \mathcal{N}(y \mid \mu_B, \sigma^2), \quad \pi_A + \pi_B = 1, \quad (40)$$

with $\mu_A < \mu_B$ and noise scale σ small relative to the mode gap $\mu_B - \mu_A$. At a fixed smoothed input, the predictor draws N samples $Y_1, \dots, Y_N \stackrel{\text{i.i.d.}}{\sim} p$, sorts them as $Y_{(1)} \leq \dots \leq Y_{(N)}$, and returns the symmetric α -trimmed mean

$$\hat{\mu}_\alpha = \frac{1}{M} \sum_{i=L+1}^{N-L} Y_{(i)}, \quad L = \lfloor \alpha N \rfloor, \quad M = N - 2L. \quad (41)$$

Each Monte Carlo realization of the smoothed predictor is one independent draw of $\hat{\mu}_\alpha$. We call the set $\{Y_{(L+1)}, \dots, Y_{(N-L)}\}$ the trimmed core.

F.2 Quantization in the noiseless limit

Let K be the (random) number of samples drawn from mode A . By construction $K \sim \text{Binom}(N, \pi_A)$. In the noiseless limit $\sigma \rightarrow 0$ the modes are perfectly separated, so after sorting the first K values come from mode A and the remaining $N - K$ from mode B . The trimmed core then contains $c_A(K) = \text{clip}(K - L, 0, M)$ values from mode A and $c_B(K) = M - c_A(K)$ from mode B . The trimmed mean then reduces to

$$\hat{\mu}_\alpha^{\sigma=0}(K) = \frac{c_A(K)\mu_A + c_B(K)\mu_B}{M}. \quad (42)$$

Since c_A takes only integer values $0, 1, \dots, M$, the predictor collapses onto $M + 1$ discrete *quantization levels*

$$\ell_j = \frac{j\mu_A + (M - j)\mu_B}{M}, \quad j = 0, 1, \dots, M, \quad (43)$$

spaced *uniformly* between μ_A and μ_B at increments of $(\mu_A - \mu_B)/M$. Each level carries the binomial probability that K takes a value mapping to it,

$$\pi_j = \begin{cases} \Pr[K \leq L] & j = 0, \\ \Pr[K = L + j] & j = 1, \dots, M - 1, \\ \Pr[K \geq N - L] & j = M. \end{cases} \quad (44)$$

The two boundary levels absorb the tails of $\text{Binom}(N, \pi_A)$ because any $K \leq L$ saturates $c_A(K) = 0$ and any $K \geq N - L$ saturates $c_A(K) = M$.

Addressing the failure mode of α smoothing. Increasing N and decreasing α help alleviate the behavior by enlarging the trimmed core, and as $N \rightarrow \infty$ or $\alpha \rightarrow 0$, the distribution approaches a continuous distribution. However, both are problematic. Decreasing α means including more outliers, and increasing N collapses the distribution to a tight peak around the mean, which is even more disjoint from the individual modes. Our clustered α -smoothing avoids the artifact by summarizing each mode with its own trimmed mean and encoding the binomial split in the partition weights $\pi^{(m)}$ instead of in the level positions.

F.3 Order-statistic correction at finite σ

The noiseless scenario highlights the origin of the quantization behavior, but neglects a bias in the level positions for $\sigma > 0$. The mode- A samples that survive trimming are not arbitrary mode- A draws but specific order statistics of the K mode- A samples (those closer to mode- B after sorting), and analogously for mode B . Both contributions are biased *toward* the gap between the modes.

Conditioning on $K = k$, the i -th overall order statistic is

$$Y_{(i)} \mid K = k \stackrel{d}{=} \begin{cases} A_{(i:k)}, & i \leq k, \\ B_{(i-k:N-k)}, & i > k, \end{cases} \quad (45)$$

where $A_{(j:n)}$ and $B_{(j:n)}$ denote the j -th order statistic among n i.i.d. draws from the corresponding mode. Writing $\nu_{j:n} = \mathbb{E}[Z_{(j:n)}]$ for the expected j -th order statistic of n standard normals, the conditional expected trimmed mean is

$$\mathbb{E}[\hat{\mu}_\alpha \mid K = k] = \frac{1}{M} [c_A(k)\mu_A + c_B(k)\mu_B + \sigma(S_A(k) + S_B(k))], \quad (46)$$

with order-statistic correction terms

$$S_A(k) = \sum_{i=L+1}^{\min(k, N-L)} \nu_{i:k} \quad (0 \text{ if } k \leq L), \quad (47)$$

$$S_B(k) = \sum_{i=\max(L+1, k+1)}^{N-L} \nu_{i-k:N-k} \quad (0 \text{ if } k \geq N-L). \quad (48)$$

Because the expected order statistic is monotone in its rank, $S_A(k)$ is non-negative (we are averaging the *upper* order statistics of the mode- A sub-sample whenever $k > L$) and $S_B(k)$ is non-positive (lower order statistics of mode B). Both shifts pull $\mathbb{E}[\hat{\mu}_\alpha \mid K = k]$ inward, i.e., toward the middle of the gap. The shifts are equal and opposite when $c_A(k) = c_B(k) = M/2$, so the central level $\ell_{M/2}$ remains at $(\mu_A + \mu_B)/2$.

The marginal expected level for the j -th quantization bin is the binomial weighted average of (46) over the k values mapping to that bin (a single k for interior bins, a tail of values for the two boundary bins).

F.4 Worked example: $N = 30$, $\alpha = 0.4$, $\pi_A = 0.4$

With these parameters, the trimmed core is $M = 6$ samples, giving $M + 1 = 7$ quantization bins. The bins are nearly uniformly spaced; the order-statistic correction compresses the outer bins inward by roughly $0.22 \approx 0.7\sigma$ and leaves the center bin invariant. Most of the probability mass concentrates at the boundary level closest to μ_B because $\mathbb{E}[K] = N\pi_A = 12 = L$, placing the median of the binomial split exactly at the lower trimming boundary.

Figure 10 shows the underlying mixture (top) alongside the empirical histogram of $\hat{\mu}_\alpha$ over 20,000 Monte Carlo realizations (bottom). The dashed lines mark the corrected levels $\bar{\ell}_j$ and labels report the binomial weights π_j from (44). Both line up tightly with the empirical peaks, confirming (43)-(46).

G Control of the residual in Proposition 3.2

One can achieve a desired residual level $\epsilon > 0$ by considering a uniform partition of the interval $[p_{\mathcal{V}_l}, \bar{p}_{\mathcal{V}_l}]$ into $K > \max_{l \in \mathcal{L}} 1 + \frac{|\mathcal{L}|}{\epsilon} (\bar{p}_{\mathcal{V}_l} - p_{\mathcal{V}_l}) \left[\frac{N}{1 - \bar{p}_{\mathcal{V}_l}} + \frac{N}{p_{\mathcal{V}_l} - p_{\mathcal{R}_l}} \right]$ anchor points. Then, it holds that $\sum_{l \in \mathcal{L}} \left[\frac{N}{1 - \bar{p}_{\mathcal{V}_l}} + \frac{N}{p_{\mathcal{V}_l} - p_{\mathcal{R}_l}} \right] h_{l,K} \leq \epsilon$.

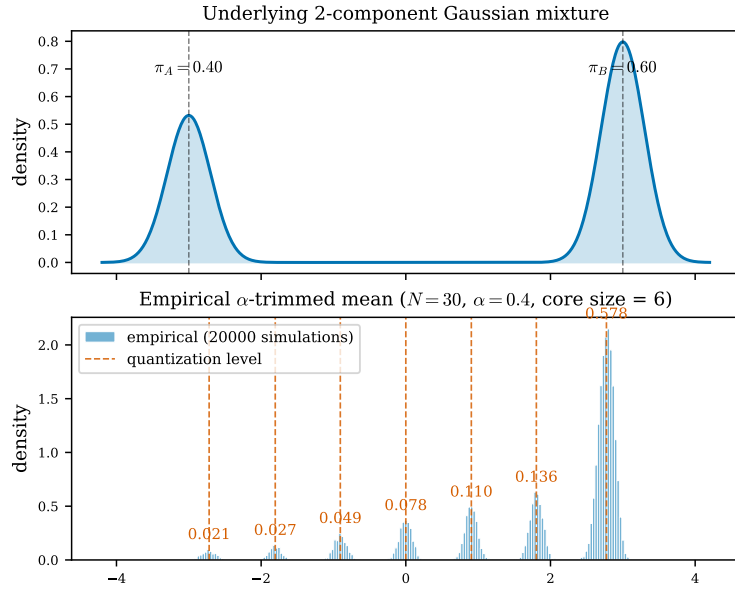


Figure 10: Quantization of the single-cluster α -trimmed predictor on a bimodal output. **Top:** the underlying mixture $0.4\mathcal{N}(-3, 0.3^2) + 0.6\mathcal{N}(+3, 0.3^2)$. **Bottom:** histogram of $\hat{\mu}_\alpha$ for $N = 30$, $\alpha = 0.4$ over 20,000 realizations. Dashed lines are the predicted quantization levels $\hat{\ell}_j$ from (46); labels are the binomial weights π_j from (44).